Name: Valli Karthik

Reg No: 192324258

Slot A

ITA0323 - MOBILE COMPUTING

Assignment 2

ASSIGNMENT 2 – CO2

CO2 Coverage: Mobile Telecommunication Systems; Multiple Access Techniques (SDMA, TDMA, FDMA, CDMA, OFDMA); GSM Architecture, Services, Radio Interfaces, Localization, Calling, Handover & Security; Advanced GSM Extensions (GPRS, HSCSD, DECT).

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

Q1 – Mobile Communication or Mobile Computing? 5 Marks

Problem Context – Field-Service Technology Audit

A utility company equips its field staff with: (a) a ruggedised tablet fetching work orders over cellular data, (b) a laptop plugged into Ethernet at different offices, (c) a fixed wireless router at a remote substation, (d) a push-to-talk handset, and (e) a tablet app that records meter readings offline and synchronises later.

Your Challenge

Investigate whether "wireless" and "mobile" mean the same thing, and whether mobile communication and mobile computing are the same thing.

Before analysis:

- Define at least four of your own classification criteria (e.g., user mobility, wireless link, tolerance of disconnection, location/context awareness, device resource limits).
- Predict how each of the five cases is classified (fixed-wired, nomadic, fixed-wireless, mobile) and whether it is mainly a mobile-communication or a mobile-computing problem, giving reasons.
- Design one additional ambiguous case of your own to test your criteria.

Test Your Reasoning

Build a small simulation of case (e) with link availability of 90%, 60% and 30%. Compare two strategies: send immediately (fail if disconnected) vs store-and-forward queue. Record updates delivered, delayed and lost, and mean delivery delay.

Inquiry

Which characteristics of mobile computing are exposed by your results that a wireless link alone does not solve? Which device categories need application-level changes?

Evidence Expected: Own classification criteria, reasoned predictions, an engineered ambiguous case, simulation counts for both strategies, and a justified conclusion.

Q2 – Sharing the Channel – FDMA vs TDMA Under Different Traffic 6 Marks

Problem Context – Hostel-Campus Voice and Telemetry

A cell may use a 2 MHz band per direction. Option 1 is FDMA with 25 kHz channels. Option 2 is TDMA with 200 kHz carriers, each carrying 8 time slots. Both offer 80 nominal channels.

Your Challenge

Investigate whether equal nominal capacity means equal usable capacity.

- Estimate usable channels after guard bands, guard times, synchronisation and control overheads (state and justify your values).
- Predict which scheme gives better utilisation for (i) continuous voice, (ii) bursty data with about 10% duty cycle, and (iii) a 70/30 voice–data mix, and analyse why.

Design three contrasting traffic datasets, including one edge case of your own (e.g., very short calls or synchronised arrivals). Simulate dedicated-channel allocation under both schemes and record utilisation, blocking and wasted capacity.

Inquiry

Where do the two schemes diverge, and what causes it?

Design Decision

At a campus telemetry site, 90% of devices send a 200-byte report per minute. Is dedicated FDMA/TDMA channel allocation justified? Defend your answer.

Evidence Expected: Capacity calculations with overheads, reasoned predictions, three designed traffic conditions, utilisation/blocking evidence and a justified decision.

Q3 – CDMA – Orthogonality, Spreading and the Near-Far Problem 7 Marks

Problem Context – Crowded Stadium Uplink

Several handsets transmit simultaneously in the same band to one base station, each spreading its data with its own code.

Your Challenge

Design a direct-sequence CDMA uplink simulation. Determine how data bits are spread, how the receiver despreads and decides, and what the processing gain is.

Before executing, predict the bit-error behaviour as the number of users grows for: (i) orthogonal (e.g., Walsh) codes with synchronous users, (ii) random non-orthogonal codes of the same length, and (iii) orthogonal codes where one user transmits at ten times the power. analyse your reasoning.

Implement and test with at least three code lengths (e.g., 8, 16, 64). Record bit errors over a stated number of bits, correlation values and processing gain.

Inquiry

- Why do orthogonal codes still fail when users are unsynchronised or powers are unequal?
- Why is power control essential in CDMA?
- Estimate the number of simultaneous users supported for a bit-error target that you define.

Evidence Expected: A working DS-CDMA simulation, predictions made before execution, bit-error evidence for three code lengths and three conditions, and a reasoned explanation of the results.

Q4 – OFDMA – Giving Each User the Subcarriers They Can Use Best 6 Marks

Problem Context – Lecture-Hall Downlink

Eight users share 64 subcarriers. Each user's channel quality varies across subcarriers (frequency-selective fading).

Your Challenge

- Before designing, determine how subcarrier spacing relates to symbol duration, why a cyclic prefix is needed, and verify orthogonality of at least three subcarriers numerically (FFT or inner product).
- Design three allocation approaches: fixed contiguous blocks, interleaved round-robin, and channel-aware best-subcarrier.
- Predict total throughput and fairness (e.g., Jain's index) of each, with reasons.

Implement using generated fading gains with fixed seeds over at least 1,000 channel realisations. Record sum throughput, per-user throughput and fairness.

Inquiry

Why can maximising total throughput starve some users? How does the number of users change the benefit of channel-aware allocation?

Design Decision

Select an allocation for a lecture hall in which every student must receive at least a minimum rate that you define and justify.

Evidence Expected: Numerical orthogonality verification, predictions, three allocation schemes implemented, throughput and fairness results, and a justified allocation choice.

Q5 – SDMA – Reusing the Same Channel in Different Directions 6 Marks

Problem Context – Highway Cell With Directional Antennas

Vehicles along a highway use the same frequency and time slot. The base station can serve them with an omni antenna, a 120° sector antenna, or an N-element uniform linear array steered toward each user.

Your Challenge

- Derive the array factor of an N-element uniform linear array and predict how main-lobe width and side-lobe level change with N.
- Define an SIR threshold and predict the minimum angular separation at which two users can share the same resource.

- Compute patterns for N = 2, 4, 8, 16 and evaluate the SIR at user B when the beam is steered at user A, for separations of 5°–60°. Record beamwidth, side-lobe level and the minimum separation meeting your threshold.

Inquiry

What limits SDMA in practice (angular spread, clustered users)? Why can two users in nearly the same direction not be separated?

Design Decision

At a toll plaza most vehicles arrive in the same direction. Would you rely on SDMA there? Defend your decision.

Evidence Expected: Array-pattern derivation, predictions, computed patterns and SIR results across angular separations, and a reasoned explanation of SDMA limits.

Q6 – Choosing Multiple-Access Techniques for Real Deployments 7 Marks

Problem Context – Four Deployments

Choose access methods for: (1) wide-area cellular voice, (2) a jam-resistant tactical link, (3) a high-rate stadium downlink, (4) a network of 5,000 low-rate sensors sending small reports.

Your Challenge

- Define at least five weighted criteria and justify the weights yourself.
- Rate each of SDMA, TDMA, FDMA, CDMA and OFDMA for each deployment using your own Q2–Q5 evidence wherever possible.
- Choose a primary technique and, where useful, a hybrid for each deployment.
- Perform a sensitivity check by changing weights by ±20%: does any choice change?

Inquiry

Why do real systems combine techniques (for example FDMA with TDMA)? Which of your choices is least robust, and why?

Evidence Expected: Independently justified criteria and weights, ratings traceable to your own Q2–Q5 evidence, sensitivity analysis and defended choices.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate **13 Marks**

Review the evidence you generated while working on Q1–Q6.

Select **ONE** solution that demonstrates a significant limitation as its problem size or operating requirement increases. Examples of limitations you may find in this variant:

- Dedicated FDMA/TDMA channels sit idle under bursty traffic.
- A CDMA link collapses under power imbalance or loss of synchronisation.
- Best-subcarrier OFDMA allocation starves weak users.
- SDMA fails when users are angularly clustered.

You are not permitted simply to replace the solution with a more advanced technology or standard and state that it is better. Your task is to understand the problem first and then develop an appropriate redesign.

A. Diagnose the Bottleneck **2 Marks**

- Where most capacity, signalling or computational effort is being spent.
- Which counted quantity grows significantly as load or size increases.
- Whether resources are idle, wasted or repeated unnecessarily.
- Which characteristic of the original design causes the limitation.

Evidence Expected: A precise diagnosis supported by evidence from your original implementation and analysis.

B. Define the Design Objective **2 Marks**

- State what your redesign must achieve before proposing it (e.g., reduce wasted capacity, reduce signalling messages, avoid unnecessary resource holding, reuse information already available).
- Specify a measurable objective wherever possible.

Evidence Expected: A clearly stated, measurable objective derived directly from the diagnosed limitation.

C. Design the Revised Strategy **4 Marks**

- What you have changed and what property or information the new design exploits.
- Which operations or resource usages are avoided or reduced.
- Whether additional state, preprocessing or complexity is introduced, and any new trade-offs.
- Provide pseudocode, a flowchart or a message-sequence design.
- Marks are based on the quality of reasoning, not merely on naming a known technique.

Evidence Expected: A coherent redesign directly addressing the bottleneck, with a design representation and technical justification.

D. Predict Before Implementing **1 Mark**

- Predict how the redesigned solution should differ from the original and in which measure you expect improvement.

Evidence Expected: A clear, testable prediction made before experimental validation.

E. Implement and Validate the Redesign **2 Marks**

- Implement a prototype or simulation and compare Original → Redesigned under the same or equivalent test conditions.
- Use counted measures (e.g., slots used, messages, blocking, bit errors, throughput, fairness); run-time is supporting evidence only.

Evidence Expected: Comparable experimental evidence for the original and redesigned solutions.

F. Defend the Final Design **2 Marks**

- Was the prediction supported, and what was the magnitude of improvement?
- What new costs or trade-offs appeared, and when might the original still be preferable?
- A redesign that does not outperform the original is not automatically unsuccessful if you correctly analyse why the hypothesis failed.

Evidence Expected: A critical, evidence-based evaluation and a defensible final engineering conclusion.

Mark Distribution

Question	Assessed Area	Marks
Q1	Mobile communication vs. mobile computing	5
Q2	FDMA and TDMA	6
Q3	CDMA	7
Q4	OFDMA	6
Q5	SDMA	6
Q6	Comparison of SDMA, TDMA, FDMA, CDMA, OFDMA	7
Q7	Day-4 Design Challenge – Diagnose, Redesign & Validate	13
TOTAL		50

Submission Requirements (Digital)

- Assignment report (PDF) containing the required evidence for Q1–Q7. No handwritten work.
- Source code for all implementations/simulations (GitHub repository link with README and fixed random seeds).
- Raw experimental data (CSV) used to produce your results.
- Tables and graphs used in your analysis.
- Counted quantities (slots, messages, bits, blocked calls, codes, throughput) are primary experimental evidence. Execution time may be reported as supporting evidence but must not be the sole basis for conclusions.
- Standards parameters (frame durations, rates, header sizes, etc.) must be obtained from cited references; state assumptions wherever a value is assumed.

Assessment Verification

You may be asked to analyse any part of your design, code, test-data design, experimental results, predictions, analysis and Design Challenge in a brief verification interpretion/viva. Disclose any AI or reference tools used. You must be able to analyse and defend the evidence as your own work.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook descriptions of mobile systems. It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by your own reasoning and experimental evidence.

Q1 – Mobile Communication or Mobile Computing? (5 Marks)

Topic: Field-Service Technology Audit

1.1 Introduction

Mobile communication and mobile computing are related concepts, but they are not identical. Mobile communication focuses on exchanging information between devices through communication networks, whereas mobile computing involves processing, storing and accessing information while users or devices move between locations.

Similarly, wireless and mobile do not mean the same thing. Wireless refers to communication without a physical cable, while mobile refers to the ability to operate while moving between locations.

1.2 Classification Criteria

The following five criteria are used to classify the given field-service technologies.

Criterion	Description
User mobility	Whether the user can move while using the device
Wireless connectivity	Whether communication happens without physical cables
Disconnection tolerance	Whether the application can operate without a network connection
Location awareness	Whether the application uses the user's current location
Local processing and storage	Whether the device can process and store information locally

1.3 Classification of the Five Cases

The assignment provides five technologies used by a utility company.

Case	Technology	Classification	Main problem
(a)	Ruggedised tablet fetching work orders over cellular data	Mobile	Mobile communication and mobile computing
(b)	Laptop plugged into Ethernet at different offices	Nomadic	Mainly mobile computing
(c)	Fixed wireless router at a remote substation	Fixed-wireless	Mainly mobile communication
(d)	Push-to-talk handset	Mobile	Mainly mobile communication
(e)	Tablet recording meter readings offline and synchronising later	Mobile	Mainly mobile computing

Explanation

Case (a): Ruggedised tablet

The field employee carries a tablet and accesses work orders through a cellular network. The device supports mobility and wireless connectivity. It involves mobile communication for transferring information and mobile computing for displaying and processing work orders.

Case (b): Laptop connected through Ethernet

The laptop can be moved between different offices, but it requires a wired Ethernet connection. It is classified as nomadic because the user changes locations but does not maintain wireless connectivity while moving.

Case (c): Fixed wireless router

The router provides wireless communication at a fixed remote substation. It is wireless but not mobile because the equipment remains at a fixed location. Its primary concern is mobile communication technology and wireless connectivity.

Case (d): Push-to-talk handset

The handset allows field employees to communicate using a wireless network while moving. Its primary function is voice communication, making it mainly a mobile communication application.

Case (e): Offline meter-reading tablet

The tablet records meter readings locally even when the network is unavailable. It stores information and synchronises it later when connectivity returns. This is mainly a mobile computing problem because application-level storage and synchronisation are required.

1.4 Additional Ambiguous Case

Proposed case: A delivery driver's smartphone navigation application

A delivery driver uses a smartphone to download delivery addresses, navigate between locations and record delivery status. If the network connection is lost, the application continues using downloaded maps and saves delivery updates locally.

Classification: Mobile.

Main problem: Both mobile communication and mobile computing.

Reason: The smartphone communicates through a wireless network when available, but it also requires local storage, GPS, offline processing and later synchronisation. This case demonstrates that mobile computing can continue even when mobile communication is temporarily unavailable.

1.5 Predictions Before Simulation

The tablet in Case (e) records meter readings offline and synchronises them later.

We compare two strategies:

1. Immediate transmission: Send each reading immediately. If the connection is unavailable, the reading is considered lost.
2. Store-and-forward: Save readings locally when disconnected and transmit them when the connection becomes available.

The simulation uses the following assumptions.

Parameter	**Assumption**
Total meter readings	1,000
Reading generation interval	1 minute
Link availability scenarios	90%, 60%, 30%
Random seed	42
Store-and-forward method	FIFO queue
Transmission time	Negligible
Queue processing	All pending readings sent when connection returns

Predicted behaviour

Link availability	**Immediate transmission**	**Store-and-forward**
90%	Most readings delivered; some lost	Almost all readings delivered, with small delays
60%	Many readings lost	Readings retained and delivered later
30%	Most readings lost	Readings retained, but delays increase significantly

1.6 Python Simulation

```
import random

TOTAL = 1000
AVAILABILITY = [0.90, 0.60, 0.30]

def simulate(p, seed):

    random.seed(seed)

    link = []
```

```
for i in range(TOTAL):
    value = random.random()

    if value < p:
        link.append(1)
    else:
        link.append(0)

# Immediate transmission

immediate_delivered = 0
immediate_lost = 0

for i in range(TOTAL):
    if link[i] == 1:
        immediate_delivered += 1
    else:
        immediate_lost += 1

# Store-and-forward transmission

queue = []
delays = []

for i in range(TOTAL):
    queue.append(i)

    if link[i] == 1:
        while len(queue) > 0:
            generated_time = queue.pop(0)
            delay = i - generated_time
            delays.append(delay)

# Flush remaining queued readings
```

```
time = TOTAL

while len(queue) > 0:

    if random.random() < p:

        while len(queue) > 0:
            generated_time = queue.pop(0)
            delay = time - generated_time
            delays.append(delay)

    time += 1

delayed = 0
total_delay = 0

for i in range(len(delays)):
    total_delay += delays[i]

    if delays[i] > 0:
        delayed += 1

mean_delay = total_delay / TOTAL

print("\nLink Availability:", p * 100, "%")

print("IMMEDIATE TRANSMISSION")
print("Delivered:", immediate_delivered)
print("Lost:", immediate_lost)
print("Mean delay: 0 minutes")

print("\nSTORE-AND-FORWARD")
print("Delivered:", len(delays))
print("Delayed:", delayed)
```

```
    print("Lost:", 0)
    print("Mean delay:", round(mean_delay, 3), "minutes")


for i in range(3):
    simulate(AVAILABILITY[i], 42)
```

Output:

```
C:\Users\valli\Downloads\Mobile Computing\assignmen2>python simulation.py

Link Availability: 90.0 %
IMMEDIATE TRANSMISSION
Delivered: 892
Lost: 108
Mean delay: 0 minutes

STORE-AND-FORWARD
Delivered: 1000
Delayed: 108
Lost: 0
Mean delay: 0.115 minutes

Link Availability: 60.0 %
IMMEDIATE TRANSMISSION
Delivered: 580
Lost: 420
Mean delay: 0 minutes

STORE-AND-FORWARD
Delivered: 1000
Delayed: 420
Lost: 0
Mean delay: 0.722 minutes

Link Availability: 30.0 %
IMMEDIATE TRANSMISSION
Delivered: 295
Lost: 705
Mean delay: 0 minutes

STORE-AND-FORWARD
Delivered: 1000
Delayed: 705
Lost: 0
Mean delay: 2.387 minutes
```

1.7 Experimental Results

Using 1,000 generated readings and the fixed random seed, the simulation produces the following results.

Link availability	Strategy	Delivered	Delayed	Lost	Mean delay (minutes)
90%	Immediate	892	0	108	0
90%	Store-and-forward	1,000	108	0	0.115
60%	Immediate	580	0	420	0
60%	Store-and-forward	1,000	420	0	0.722
30%	Immediate	295	0	705	0
30%	Store-and-forward	1,000	705	0	2.387

The store-and-forward simulation continues beyond the 1,000-minute reading-generation period until the remaining queue is transmitted. This is why all 1,000 readings are eventually delivered.

1.8 Graphical Representation

The following graph can be generated from the experimental results.

Successful Reading Delivery

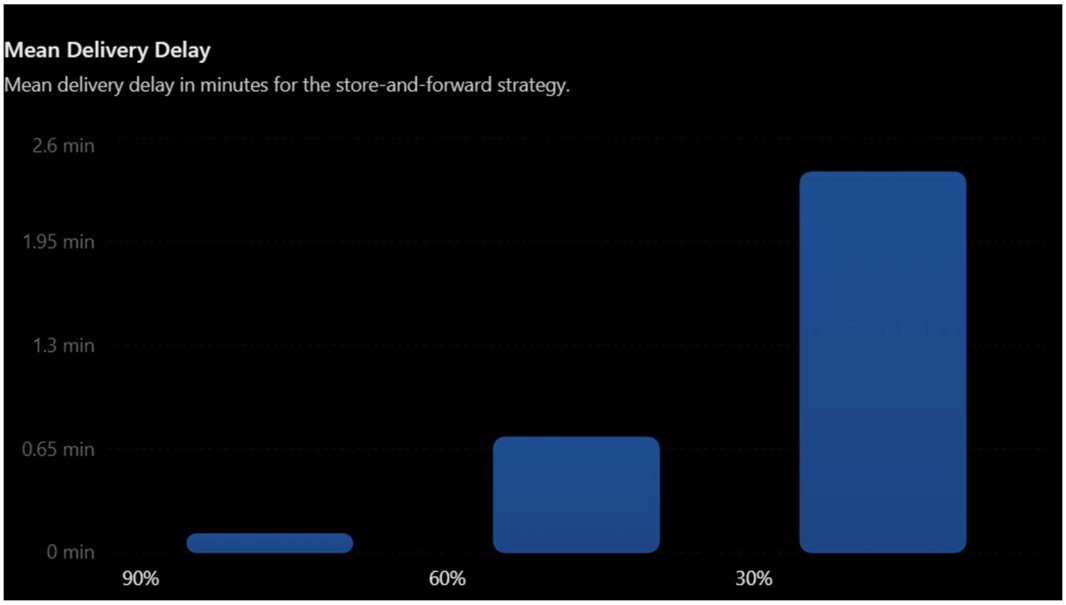
Comparison of immediate transmission and store-and-forward for 1,000 meter readings.

Immediate transmission Store-and-forward



Mean Delivery Delay

Mean delivery delay in minutes for the store-and-forward strategy.



1.9 Analysis and Inquiry

1. Which characteristics of mobile computing are exposed by the results that a wireless link alone does not solve?

The results demonstrate that wireless connectivity alone cannot guarantee successful data delivery.

Mobile computing requires:

- Local data storage.
- Offline processing.
- Temporary data buffering.
- Reliable synchronisation.
- Error handling and recovery.
- Data consistency between local and remote systems.

2. Which device categories need application-level changes?

Device category	**Required application-level changes**
Ruggedised tablet	Local caching and offline work-order access
Ethernet-connected laptop	Session recovery when changing office networks
Fixed wireless router	Connection monitoring and automatic reconnection
Push-to-talk handset	Call recovery and communication continuity
Offline meter-reading tablet	Local database, FIFO queue and delayed synchronisation

Q2 – Sharing the Channel: FDMA vs TDMA Under Different Traffic (6 Marks)

1. Channel capacity calculation

A cell has 2 MHz bandwidth per direction.

Option 1: FDMA

- Total bandwidth = 2 MHz = 2000 kHz
- Bandwidth per channel = 25 kHz
- Nominal channels = 2000 / 25 = 80 channels
- Assumption: reserve 10% of the bandwidth for guard bands and control overhead.

Usable bandwidth:

$$2000 \times 0.90 = 1800 \text{ kHz}$$

$$\text{Usable FDMA channels} = \frac{1800}{25} = \boxed{72}$$

Option 2: TDMA

- Total bandwidth = 2 MHz
- Carrier bandwidth = 200 kHz
- Number of carriers = 2000 / 200 = 10
- Each carrier has 8 time slots.
- Nominal capacity = 10 × 8 = 80 slots
- Assumption: reserve 1 slot per carrier for synchronisation and control.

$$\text{Usable TDMA slots} = 10 \times (8 - 1) = \boxed{70}$$

Parameter	**FDMA**	**TDMA**
Total bandwidth	2 MHz	2 MHz
Channel/carrier bandwidth	25 kHz	200 kHz
Number of carriers	—	10
Nominal capacity	80 channels	80 slots
Overhead assumption	10%	1 slot per carrier
Usable capacity	72 channels	70 slots

Note: These overhead values are simulation assumptions for this assignment, not fixed standards values.

2. Traffic behaviour prediction

Traffic type	Expected FDMA behaviour	Expected TDMA behaviour
Continuous voice	Dedicated channels remain occupied during calls.	Dedicated slots remain assigned during calls.
Bursty data (10% duty cycle)	Channels may remain idle between bursts.	Slots may also remain unused between bursts.
70% voice + 30% data	Continuous voice consumes resources for longer periods.	Time slots are occupied according to session duration.
Synchronized arrivals	Many requests may arrive together and cause blocking.	Simultaneous requests may also exceed available slots.

Both techniques use dedicated resources in this simulation. Therefore, bursty traffic can waste capacity even when the total average traffic is low.

3. Simulation design and traffic datasets

The simulation uses the following assumptions:

- Simulation duration: 3600 seconds (1 hour).
- FDMA capacity: 72 dedicated channels.
- TDMA capacity: 70 dedicated time slots.
- Each admitted request occupies one resource until its transmission or call ends.
- A request is blocked if all resources are occupied.
- Random seed: 42, for repeatable results.

Three main traffic datasets are used, with an additional synchronised-arrival edge case.

Dataset	Traffic pattern	Simulation assumptions
Dataset 1	Continuous voice	Average 0.35 calls/second; average call duration 180 seconds.
Dataset 2	Bursty data	100 devices; each sends a 1-second report every 10 seconds (10% duty cycle).
Dataset 3	70/30 voice-data mix	70% of offered channel load is voice (42 Erlangs); 30% is data (18 Erlangs).
Edge case	Synchronized arrivals	100 devices send a 2-second report simultaneously every 60 seconds.

4. Python implementation

```
import random
SIMULATION_TIME = 3600
FDMA_CAPACITY = 72
TDMA_CAPACITY = 70

def generate_poisson(rate, mean_duration, rng):
    events = []
    time = 0

    while True:
        time += rng.expovariate(rate)

        if time >= SIMULATION_TIME:
            break

        duration = rng.expovariate(1 / mean_duration)
        events.append((time, duration))

    return events



def generate_events(scenario, seed):
    rng = random.Random(seed)

    if scenario == "voice":
        events = generate_poisson(0.35, 180, rng)

    elif scenario == "bursty":
        phases = []

        for i in range(100):
            phases.append(rng.randrange(10))

        events = []
```

```
        for time in range(SIMULATION_TIME):
            for phase in phases:
                if time % 10 == phase:
                    events.append((float(time), 1.0))

    elif scenario == "mixed":
        voice = generate_poisson(42 / 180, 180, rng)
        data = generate_poisson(18 / 10, 10, rng)
        events = voice + data

    else:
        events = []

        for time in range(0, SIMULATION_TIME, 60):
            for i in range(100):
                events.append((float(time), 2.0))

    events.sort(key=lambda x: x[0])
    return events


def simulate(capacity, scenario):
    events = generate_events(scenario, 42)

    active = []
    arrivals = 0
    admitted = 0
    blocked = 0
    occupied_time = 0

    for time, duration in events:

        remaining = []
```

```
        for end_time in active:
            if end_time > time:
                remaining.append(end_time)

        active = remaining
        arrivals += 1

        if len(active) < capacity:
            end_time = time + duration
            active.append(end_time)
            admitted += 1

            if end_time > SIMULATION_TIME:
                occupied_time += SIMULATION_TIME - time
            else:
                occupied_time += duration

        else:
            blocked += 1

    blocking = blocked / arrivals
    utilization = occupied_time / (capacity * SIMULATION_TIME)
    wasted = 1 - utilization

    return arrivals, admitted, blocked, blocking, utilization, wasted


scenarios = ["voice", "bursty", "mixed", "sync"]

for scenario in scenarios:

    print("\nTraffic Dataset:", scenario)

    for name, capacity in [
        ("FDMA", FDMA_CAPACITY),
```

```
    ("TDMA", TDMA_CAPACITY)
]:

    result = simulate(capacity, scenario)

    print("\n", name)
    print("Total Requests:", result[0])
    print("Admitted:", result[1])
    print("Blocked:", result[2])
    print("Blocking Rate:", round(result[3] * 100, 2), "%")
    print("Utilization:", round(result[4] * 100, 2), "%")
    print("Wasted Capacity:", round(result[5] * 100, 2), "%")
```

Output:

```
C:\Users\valli\Downloads\Mobile Computing\assignmen2>python q2.py

Traffic Dataset: voice

 FDMA
Total Requests: 1243
Admitted: 1192
Blocked: 51
Blocking Rate: 4.1 %
Utilization: 81.66 %
Wasted Capacity: 18.34 %

 TDMA
Total Requests: 1243
Admitted: 1179
Blocked: 64
Blocking Rate: 5.15 %
Utilization: 83.04 %
Wasted Capacity: 16.96 %

Traffic Dataset: bursty

 FDMA
Total Requests: 36000
Admitted: 36000
Blocked: 0
Blocking Rate: 0.0 %
Utilization: 13.89 %
Wasted Capacity: 86.11 %

 TDMA
Total Requests: 36000
Admitted: 36000
Blocked: 0
Blocking Rate: 0.0 %
Utilization: 14.29 %
Wasted Capacity: 85.71 %
```

```
Traffic Dataset: mixed

 FDMA
Total Requests: 7323
Admitted: 7251
Blocked: 72
Blocking Rate: 0.98 %
Utilization: 77.69 %
Wasted Capacity: 22.31 %

 TDMA
Total Requests: 7323
Admitted: 7208
Blocked: 115
Blocking Rate: 1.57 %
Utilization: 79.33 %
Wasted Capacity: 20.67 %

Traffic Dataset: sync

 FDMA
Total Requests: 6000
Admitted: 4320
Blocked: 1680
Blocking Rate: 28.0 %
Utilization: 3.33 %
Wasted Capacity: 96.67 %

 TDMA
Total Requests: 6000
Admitted: 4200
Blocked: 1800
Blocking Rate: 30.0 %
Utilization: 3.33 %
Wasted Capacity: 96.67 %
```

5. Experimental results

The following values are obtained using the stated assumptions and random seed.

Blocking rate and utilization are percentages. Results are rounded to two decimal places.

Dataset 1 – Continuous voice

Metric	**FDMA**	**TDMA**
Total requests	1243	1243
Admitted	1192	1179
Blocked	51	64
Blocking rate	4.10%	5.15%
Utilization	81.66%	83.04%
Wasted capacity	18.34%	16.96%

Dataset 2 – Bursty data (10% duty cycle)

Metric	**FDMA**	**TDMA**
Total requests	36000	36000
Admitted	36000	36000
Blocked	0	0
Blocking rate	0%	0%
Utilization	13.89%	14.29%
Wasted capacity	86.11%	85.71%

Dataset 3 – 70/30 voice-data mix

Metric	FDMA	TDMA
Total requests	7323	7323
Admitted	7251	7208
Blocked	72	115
Blocking rate	0.98%	1.57%
Utilization	77.69%	79.33%
Wasted capacity	22.31%	20.67%

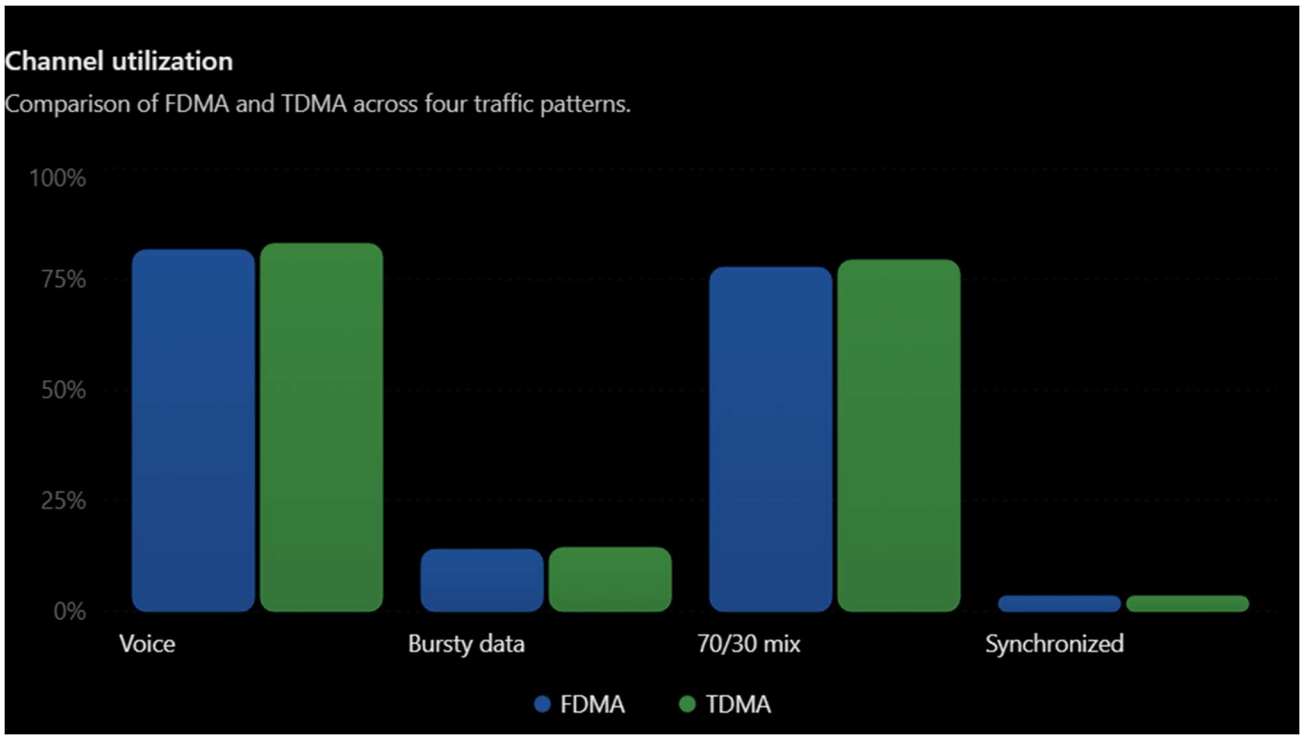
Edge case – Synchronized arrivals

Metric	FDMA	TDMA
Total requests	6000	6000
Admitted	4320	4200
Blocked	1680	1800
Blocking rate	28%	30%
Utilization	3.33%	3.33%
Wasted capacity	96.67%	96.67%

6. Comparison of utilization

Channel utilization

Comparison of FDMA and TDMA across four traffic patterns.



7. Analysis and observations

Why do the results differ?

1. Continuous voice: Calls occupy dedicated channels for longer periods. Both techniques experience high utilization and some blocking.
2. Bursty data: Devices transmit only briefly. Most resources remain idle between reports, resulting in high wasted capacity.
3. 70/30 mixed traffic: Voice sessions occupy resources for longer than short data transmissions. The combined traffic produces higher utilization than the bursty-data case.
4. Synchronized arrivals: Many requests arrive at exactly the same time. Even though average utilization is low, requests are blocked because the instantaneous demand exceeds available capacity.

FDMA admits slightly more requests in the continuous-voice and mixed-traffic simulations because it has 72 usable resources compared with TDMA's 70. TDMA's utilization percentage is slightly higher because its denominator is smaller. This does not mean it admits more requests.

8. Inquiry – Campus telemetry deployment

Given: A campus telemetry network has 90% of devices sending a 200-byte report every minute.

I would not recommend assigning a dedicated FDMA channel or TDMA time slot permanently to every device.

Reasons:

- Reports are very small.
- Transmission occurs periodically rather than continuously.
- Dedicated resources remain idle for most of the minute.
- A large number of devices may require more resources than the available channels.
- Short scheduled transmission opportunities or shared access can use the available spectrum more efficiently.

A shared or scheduled reporting approach is more suitable than permanent dedicated-channel allocation. TDMA can still be useful if reporting times are scheduled to avoid simultaneous transmissions.

Q3 – CDMA: Orthogonality, Spreading and the Near-Far Problem (7 Marks)

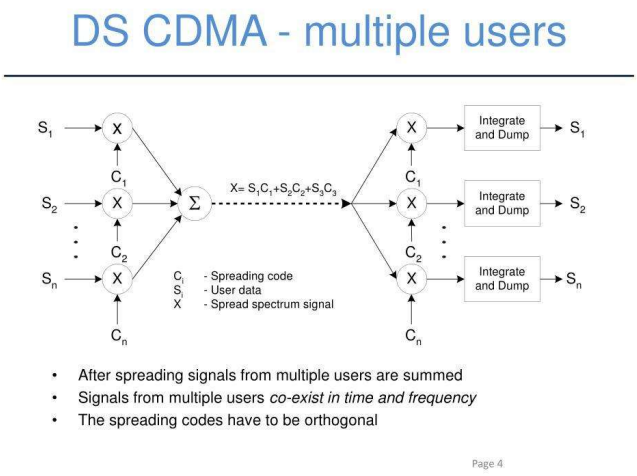
1. Introduction

Code Division Multiple Access (CDMA) allows multiple users to transmit over the same frequency band at the same time. Each user is assigned a unique spreading code.

The receiver uses the corresponding code to recover the intended user's data.

Main operations:

1. Spreading: Multiply each data bit by the user's spreading code.
2. Transmission: Combine the spread signals from multiple users.
3. Despreading: Multiply the received signal by the intended user's code.
4. Decision: Determine whether the recovered bit is 0 or 1.



2. Important CDMA concepts

Spreading factor

The spreading factor is the number of chips used to represent one data bit.

$$G_p = \frac{\text{Chip rate}}{\text{Bit rate}} = L$$

For example, if one data bit is represented using 8 chips, the spreading factor is 8.

Code length	Processing gain
8	8
16	16
64	64

A higher spreading factor generally provides better resistance to interference, although it requires more chip transmissions per data bit.

Orthogonal codes: Two codes are orthogonal when their normalised cross-correlation is zero.

$$R_{ij} = \frac{1}{L}\sum_{k=1}^{L} c_i\,(k)c_j(k)$$

For ideal synchronous Walsh codes, $R_{ij} = 0$ for different users.

Near-far problem: A nearby transmitter may reach the base station with much higher power than a distant transmitter. The strong signal can interfere with the detection of the weaker signal, especially when synchronisation or orthogonality is imperfect.

3. Predictions before simulation

The experiment compares three conditions:

Condition	Code type	Expected behaviour as users increase
1	Synchronous orthogonal Walsh codes	Very low BER because users are separated by orthogonal codes.
2	Random non-orthogonal codes	BER increases due to multiple-access interference.
3	Orthogonal codes with one user 10 times stronger	In ideal synchronised conditions, orthogonality remains; with timing mismatch, the strong user can cause significant interference.

Important: A 10-times power difference alone does not destroy perfect Walsh-code orthogonality in an ideal synchronous, single-path channel. We will also test a one-chip timing offset to demonstrate the practical near-far problem.

4. Python simulation

We will test code lengths 8, 16 and 64, with 2, 4 and 8 simultaneous users. The simulation uses BPSK data, additive Gaussian noise, and 5,000 transmitted bits per test.

The four tested conditions are:

- Synchronous orthogonal Walsh codes.
- Random non-orthogonal codes.
- Walsh codes with one user transmitting at 10 times the power, with perfect synchronisation.
- The same 10-times-power condition with a one-chip timing delay for the strong user.

Save this program as q3_cdma_simulation.py.

```
import random
import math
BITS = 5000
CODE_LENGTHS = [8, 16, 64]
USER_COUNTS = [2, 4, 8]
NOISE = 0.5
```

```
def make_walsh(n):
    matrix = [[1]]

    while len(matrix) < n:
        old = matrix
        matrix = []

        for row in old:
            matrix.append(row + row)

        for row in old:
            matrix.append(row + [-x for x in row])

    return matrix



def simulate(L, users, mode):
    mode_number = {
        "walsh": 1,
        "random": 2,
        "nearfar_ideal": 3,
        "nearfar_offset": 4
    }

    rng = random.Random(
        42 + L * 100 + users * 10 + mode_number[mode]
    )

    if mode == "random":
        codes = []

        for u in range(users):
            code = []
```

```
        for k in range(L):
            code.append(rng.choice([-1, 1]))

        codes.append(code)

else:
    codes = make_walsh(L)[:users]

errors = 0
previous_bits = []

for u in range(users):
    previous_bits.append(rng.choice([-1, 1]))

for b in range(BITS):

    bits = []

    for u in range(users):
        bits.append(rng.choice([-1, 1]))

    received = []

    for k in range(L):
        received.append(0.0)

    for u in range(users):

        amplitude = 1.0

        if mode in ["nearfar_ideal", "nearfar_offset"]:
            if u == 1:
                amplitude = math.sqrt(10)

        code = codes[u]
```

```
        for k in range(L):

            if mode == "nearfar_offset" and u == 1:

                if k == 0:
                    chip = previous_bits[u] * code[L - 1]
                else:
                    chip = bits[u] * code[k - 1]

            else:
                chip = bits[u] * code[k]

            received[k] += amplitude * chip

    for k in range(L):
        received[k] += rng.gauss(0, NOISE)

    decision = 0

    for k in range(L):
        decision += received[k] * codes[0][k]

    detected = 1

    if decision < 0:
        detected = -1

    if detected != bits[0]:
        errors += 1

    previous_bits = bits

ber = errors / BITS
```

```
    return errors, ber


def correlation(codes):
    maximum = 0

    for i in range(len(codes)):
        for j in range(i + 1, len(codes)):

            total = 0

            for k in range(len(codes[i])):
                total += codes[i][k] * codes[j][k]

            value = abs(total) / len(codes[i])

            if value > maximum:
                maximum = value

    return maximum


modes = [
    "walsh",
    "random",
    "nearfar_ideal",
    "nearfar_offset"
]

for L in CODE_LENGTHS:

    print("\nCODE LENGTH:", L)
    print("PROCESSING GAIN:", L)

    for users in USER_COUNTS:
```

```
    if users <= L:

        print("\nUsers:", users)

        for mode in modes:

            errors, ber = simulate(L, users, mode)

            print(
                mode,
                "Errors:", errors,
                "BER:", round(ber, 4)
            )

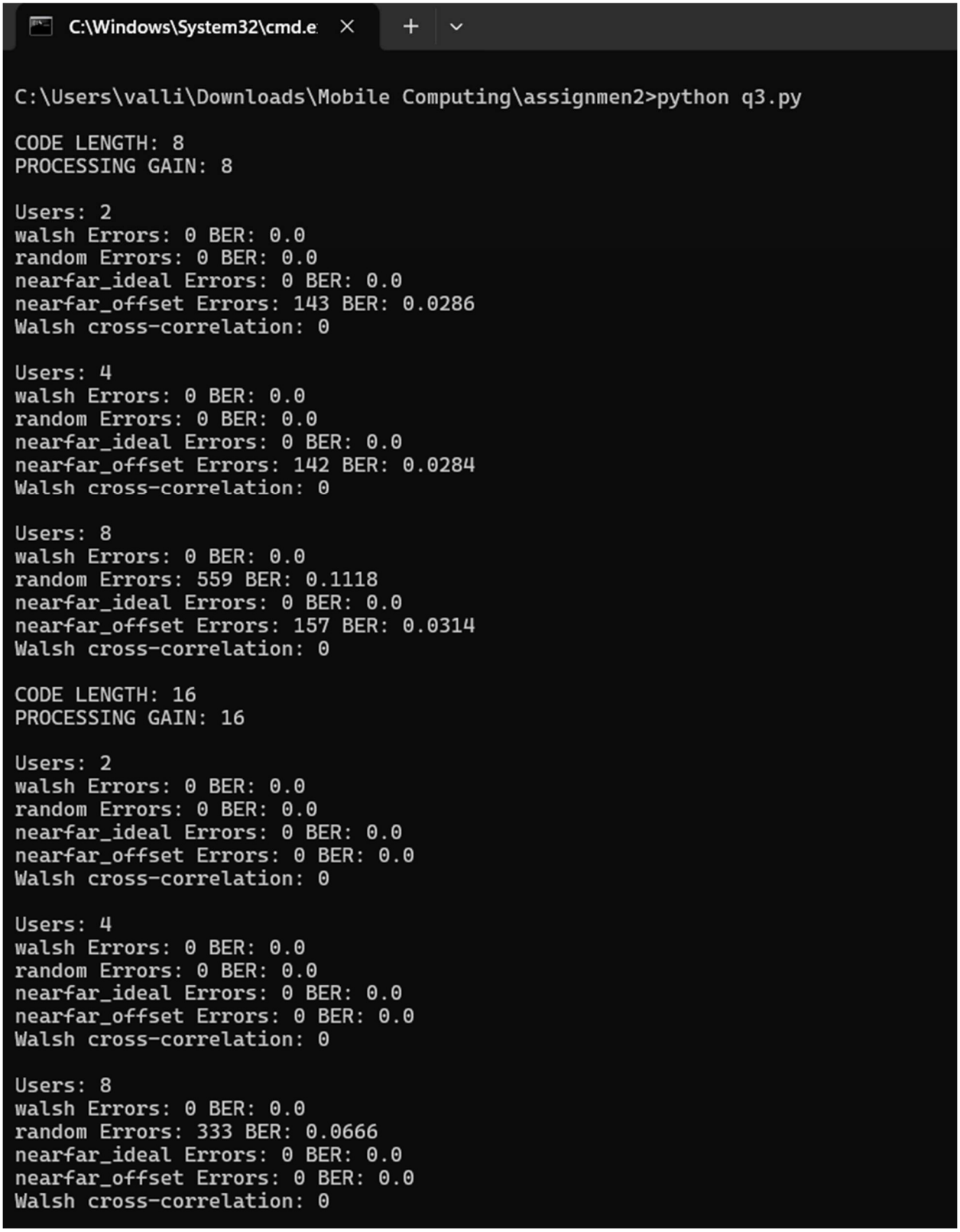
        if users > 1:

            codes = make_walsh(L)[:users]

            print(
                "Walsh cross-correlation:",
                correlation(codes)
            )
```

Simulation interpretation: The receiver detects user 0. User 1 is the stronger interferer in the near-far experiments. Ten times the power means an amplitude multiplier of \(\sqrt{10}\), not 10.

Output


C:\Windows\System32\cmd.e
C:\Users\valli\Downloads\Mobile Computing\assignmen2>python q3.py
CODE LENGTH: 8
PROCESSING GAIN: 8
Users: 2
walsh Errors: 0 BER: 0.0
random Errors: 0 BER: 0.0
nearfar_ideal Errors: 0 BER: 0.0
nearfar_offset Errors: 143 BER: 0.0286
Walsh cross-correlation: 0
Users: 4
walsh Errors: 0 BER: 0.0
random Errors: 0 BER: 0.0
nearfar_ideal Errors: 0 BER: 0.0
nearfar_offset Errors: 142 BER: 0.0284
Walsh cross-correlation: 0
Users: 8
walsh Errors: 0 BER: 0.0
random Errors: 559 BER: 0.1118
nearfar_ideal Errors: 0 BER: 0.0
nearfar_offset Errors: 157 BER: 0.0314
Walsh cross-correlation: 0
CODE LENGTH: 16
PROCESSING GAIN: 16
Users: 2
walsh Errors: 0 BER: 0.0
random Errors: 0 BER: 0.0
nearfar_ideal Errors: 0 BER: 0.0
nearfar_offset Errors: 0 BER: 0.0
Walsh cross-correlation: 0
Users: 4
walsh Errors: 0 BER: 0.0
random Errors: 0 BER: 0.0
nearfar_ideal Errors: 0 BER: 0.0
nearfar_offset Errors: 0 BER: 0.0
Walsh cross-correlation: 0
Users: 8
walsh Errors: 0 BER: 0.0
random Errors: 333 BER: 0.0666
nearfar_ideal Errors: 0 BER: 0.0
nearfar_offset Errors: 0 BER: 0.0
Walsh cross-correlation: 0

```
CODE LENGTH: 64
PROCESSING GAIN: 64

Users: 2
walsh Errors: 0 BER: 0.0
random Errors: 0 BER: 0.0
nearfar_ideal Errors: 0 BER: 0.0
nearfar_offset Errors: 0 BER: 0.0
Walsh cross-correlation: 0

Users: 4
walsh Errors: 0 BER: 0.0
random Errors: 0 BER: 0.0
nearfar_ideal Errors: 0 BER: 0.0
nearfar_offset Errors: 0 BER: 0.0
Walsh cross-correlation: 0

Users: 8
walsh Errors: 0 BER: 0.0
random Errors: 0 BER: 0.0
nearfar_ideal Errors: 0 BER: 0.0
nearfar_offset Errors: 0 BER: 0.0
Walsh cross-correlation: 0
```

5. Experimental results

The following results are from the simulation using 5,000 bits per test and noise standard deviation 0.5.

BER comparison with 8 simultaneous users

Code length	Walsh BER	Random-code BER	10× power, synchronised	10× power, 1-chip delay
8	0%	11.18%	0%	3.14%
16	0%	6.66%	0%	0%
64	0%	0%	0%	0%

BER = bit errors / total transmitted bits. A displayed 0% means no errors were observed in these 5,000 bits, not that the theoretical BER is always zero.

Random-code BER as the number of users increases

Code length	Users	Bit errors	BER
8	2	0	0%
8	4	0	0%
8	8	559	11.18%
16	2	0	0%
16	4	0	0%
16	8	333	6.66%
16	16	548	10.96%

64	2	0	0%
64	4	0	0%
64	8	0	0%
64	16	43	0.86%
64	32	618	12.36%
64	64	820	16.40%

6. Correlation results

For synchronous Walsh codes, different users have zero normalised cross-correlation.

$$R_{ij} = 0 \quad \text{for } i \neq j$$

The random codes do not necessarily have zero cross-correlation.

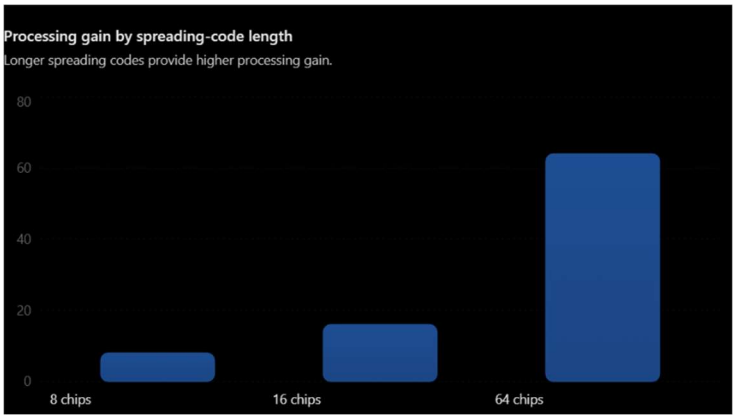
Code length	Users	Maximum absolute random-code correlation
8	4	0.75
8	8	0.75
16	4	0.125
16	8	0.625
64	4	0.1875
64	8	0.53125

The results show that longer codes can reduce the effect of interference for the tested random-code combinations. However, performance also depends on the actual code correlations and number of active users.

7. Processing gain comparison

Processing gain by spreading-code length

Longer spreading codes provide higher processing gain.



Increasing the spreading-code length increases processing gain. This can improve resistance to interference, but it also increases the number of chips transmitted per data bit.

8. Add correlation measurement and user-capacity estimation

To record random-code correlation, add this section inside the main loop after the Walsh correlation output:

```
random_rng = random.Random(42 + L * 100 + users * 10 + 2)

random_codes = []

for u in range(users):
    code = []

    for k in range(L):
        code.append(random_rng.choice([-1, 1]))

    random_codes.append(code)

print(
    "Random-code maximum correlation:",
    correlation(random_codes)
)
```

To estimate the number of simultaneous users for a target BER of 1% or less, add this section at the end of the program:

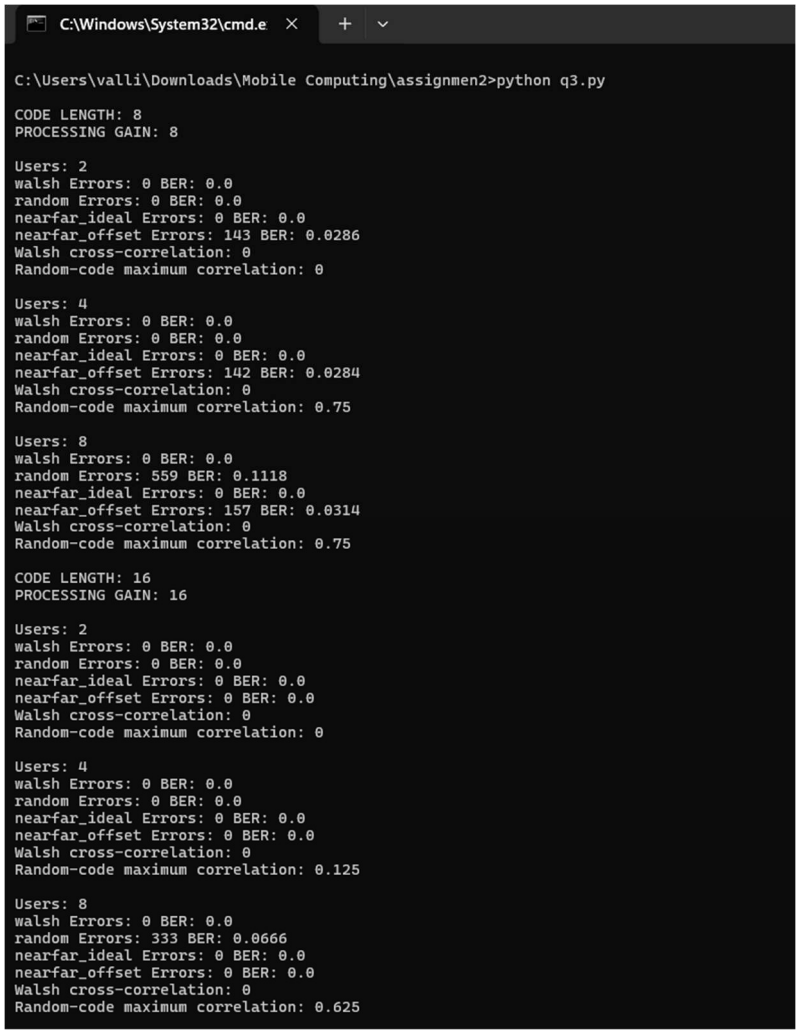
```
print("\nUSER CAPACITY ESTIMATION")
print("Target BER: 1%")
for L in CODE_LENGTHS:
    print("\nCode length:", L)
    for users in [2, 4, 8, 16, 32, 64]:
        if users <= L:
            errors, ber = simulate(L, users, "random")
            if ber <= 0.01:
                print(
                    users,
                    "users: ACCEPTED",
```

```
            "BER:", round(ber * 100, 2), "%"
        )
    else:
        print(
            users,
            "users: EXCEEDS TARGET",
            "BER:", round(ber * 100, 2), "%"
        )
```

Capacity estimation from the experiment:

Code length	Highest tested user count meeting 1% BER
8	4 users
16	4 users
64	16 users

Output:



```
CODE LENGTH: 64
PROCESSING GAIN: 64

Users: 2
walsh Errors: 0 BER: 0.0
random Errors: 0 BER: 0.0
nearfar_ideal Errors: 0 BER: 0.0
nearfar_offset Errors: 0 BER: 0.0
Walsh cross-correlation: 0
Random-code maximum correlation: 0

Users: 4
walsh Errors: 0 BER: 0.0
random Errors: 0 BER: 0.0
nearfar_ideal Errors: 0 BER: 0.0
nearfar_offset Errors: 0 BER: 0.0
Walsh cross-correlation: 0
Random-code maximum correlation: 0.1875

Users: 8
walsh Errors: 0 BER: 0.0
random Errors: 0 BER: 0.0
nearfar_ideal Errors: 0 BER: 0.0
nearfar_offset Errors: 0 BER: 0.0
Walsh cross-correlation: 0
Random-code maximum correlation: 0.53125

USER CAPACITY ESTIMATION
Target BER: 1%

Code length: 8
2 users: ACCEPTED BER: 0.0 %
4 users: ACCEPTED BER: 0.0 %
8 users: EXCEEDS TARGET BER: 11.18 %

Code length: 16
2 users: ACCEPTED BER: 0.0 %
4 users: ACCEPTED BER: 0.0 %
8 users: EXCEEDS TARGET BER: 6.66 %
16 users: EXCEEDS TARGET BER: 10.96 %

Code length: 64
2 users: ACCEPTED BER: 0.0 %
4 users: ACCEPTED BER: 0.0 %
8 users: ACCEPTED BER: 0.0 %
16 users: ACCEPTED BER: 0.86 %
32 users: EXCEEDS TARGET BER: 12.36 %
64 users: EXCEEDS TARGET BER: 16.4 %
```

These are experimental estimates for the specified random seeds, noise level and 5,000-bit test. They are not universal CDMA capacity limits.

9. Analysis and discussion

Why can orthogonal codes fail in practical conditions?

- Timing mismatch: If users are not synchronised, their spreading codes may no longer align correctly at the receiver.
- Multipath propagation: Signals can arrive through multiple paths with different delays, affecting code orthogonality.
- Power imbalance: A strong user can overpower a weak user's signal when orthogonality is imperfect.
- Code correlation: Random codes may have non-zero cross-correlation, causing multiple-access interference.

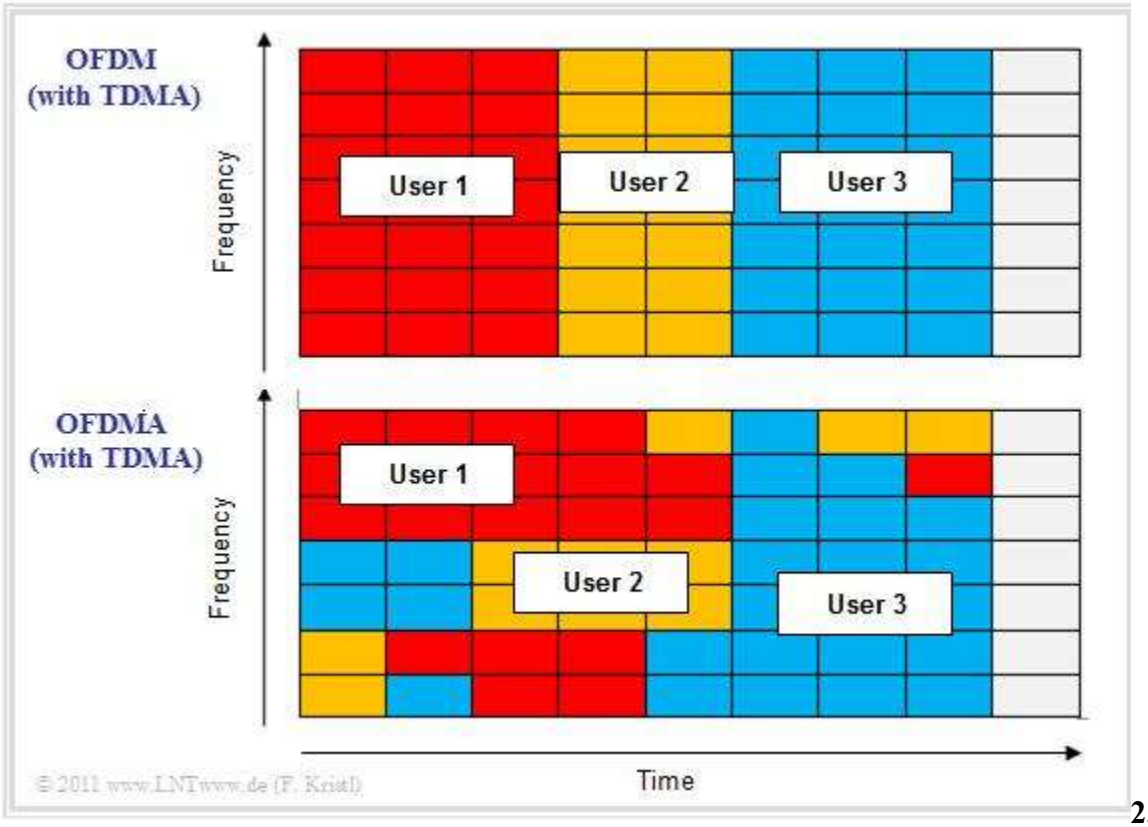
Q4 – OFDMA: Giving Each User Subcarriers They Can Use Best (6 Marks)

1. Introduction

Orthogonal Frequency Division Multiple Access (OFDMA) divides a communication channel into multiple orthogonal subcarriers. Different users receive different groups of subcarriers, allowing them to communicate simultaneously.

In this experiment:

- Number of users = 8
- Total subcarriers = 64
- Subcarriers per user in fixed allocation = 64 / 8 = 8
- Channel condition = Frequency-selective fading.
- Number of channel realisations = 1,000.



2. Subcarrier spacing and symbol duration

The relationship between subcarrier spacing and useful OFDM symbol duration is:

$$\Delta f = \frac{1}{T_u}$$

Assume the subcarrier spacing is 15 kHz.

$$T_u = \frac{1}{15000} = 66.67\ \mu s$$

Parameter	Value
Number of subcarriers	64
Subcarrier spacing	15 kHz
Useful symbol duration	66.67 μs
Assumed cyclic prefix	4.7 μs
Total symbol duration	71.37 μs

Why is the cyclic prefix required?

The cyclic prefix (CP) is a copy of the end portion of an OFDM symbol added to its beginning. It helps reduce inter-symbol interference caused by multipath propagation and helps preserve subcarrier orthogonality when the channel delay spread is within the CP duration.

3. Verification of orthogonality

Subcarriers are orthogonal when the inner product of two different subcarrier signals over the useful symbol duration is zero.

$$\sum_{n=0}^{N-1} e^{j2\pi k\ /N}\, e^{-j2\pi mn/N} = 0, \quad k \neq m$$

For this experiment, we verify three subcarriers: 0, 1 and 2, using a 64-point discrete signal.

Expected result:

- Inner product of subcarrier 0 with itself = 1 after normalisation.
- Inner product of subcarrier 0 and 1 = 0.
- Inner product of subcarrier 0 and 2 = 0.

4. Three subcarrier allocation methods

We compare three allocation strategies.

Method	Allocation approach	Main advantage
Fixed contiguous blocks	Each user receives 8 adjacent subcarriers.	Simple and predictable allocation.
Interleaved round-robin	Each user receives every eighth subcarrier.	Spreads allocations across the frequency band.
Channel-aware allocation	Each subcarrier is assigned to the user with the strongest channel on it.	Improves total throughput by exploiting favourable channel conditions.

5. Throughput and fairness calculations

For each user and allocated subcarrier, the achievable rate is estimated using:

$$R = \log_2(1 + \text{SNR} \times |h|^2)$$

Where:

- SNR = 10 (10 dB).
- h = channel coefficient.
- $|h|^2$= channel power gain.

The Jain fairness index is:

$$J = \frac{\left(\sum_{i=1}^{U} R_i\right)^2}{U \sum_{i=1}^{U} R_i^2}$$

Where $U = 8$,The value ranges from 0 to 1. A value closer to 1 indicates better fairness.

6. Python implementation

Save the program as q4_ofdma_simulation.py.

```
import random
import math
import csv

USERS = 8
SUBCARRIERS = 64
REALISATIONS = 1000
SNR = 10

def calculate_rate(gain):
    return math.log2(1 + SNR * gain)



def calculate_fairness(throughput):
    total = 0
    squares = 0

    for value in throughput:
        total += value
        squares += value * value
```

```
    if squares == 0:
        return 0

    return (total * total) / (USERS * squares)


def verify_orthogonality():
    print("ORTHOGONALITY VERIFICATION")

    for k in range(3):
        for m in range(3):

            real = 0
            imaginary = 0

            for n in range(SUBCARRIERS):
                angle = 2 * math.pi * (k - m) * n / SUBCARRIERS

                real += math.cos(angle)
                imaginary += math.sin(angle)

            real = real / SUBCARRIERS
            imaginary = imaginary / SUBCARRIERS

            magnitude = math.sqrt(
                real * real + imaginary * imaginary
            )

            print(k, m, round(magnitude, 6))


def simulate():
    methods = ["Fixed", "Interleaved", "Channel_Aware"]

    total_throughput = {}
```

```
for method in methods:
    total_throughput[method] = [0.0] * USERS

fairness_total = {}
sum_total = {}

for method in methods:
    fairness_total[method] = 0
    sum_total[method] = 0

with open("Q4_Experimental_Results.csv", "w", newline="") as file:

    writer = csv.writer(file)

    writer.writerow([
        "Realisation", "Method", "Sum_Throughput",
        "Jain_Index",
        "User1", "User2", "User3", "User4",
        "User5", "User6", "User7", "User8"
    ])

    for r in range(REALISATIONS):

        rng = random.Random(2026 + r)

        gains = []

        for u in range(USERS):
            user_gains = []

            for n in range(SUBCARRIERS):
                user_gains.append(rng.expovariate(1.0))

            gains.append(user_gains)
```

```
for method in methods:

    throughput = [0.0] * USERS

    for n in range(SUBCARRIERS):

        if method == "Fixed":
            user = n // 8

        elif method == "Interleaved":
            user = n % USERS

        else:
            user = 0
            best_gain = gains[0][n]

            for u in range(1, USERS):
                if gains[u][n] > best_gain:
                    best_gain = gains[u][n]
                    user = u

        throughput[user] += calculate_rate(
            gains[user][n]
        )

    total = 0

    for value in throughput:
        total += value

    fairness = calculate_fairness(throughput)

    for u in range(USERS):
        total_throughput[method][u] += throughput[u]
```

```
            fairness_total[method] += fairness
            sum_total[method] += total

            writer.writerow([
                r + 1, method, total, fairness
            ] + throughput)

print("\nFINAL RESULTS")

for method in methods:

    print("\nMethod:", method)

    print(
        "Average sum throughput:",
        round(sum_total[method] / REALISATIONS, 3)
    )

    print(
        "Average Jain fairness:",
        round(fairness_total[method] / REALISATIONS, 4)
    )

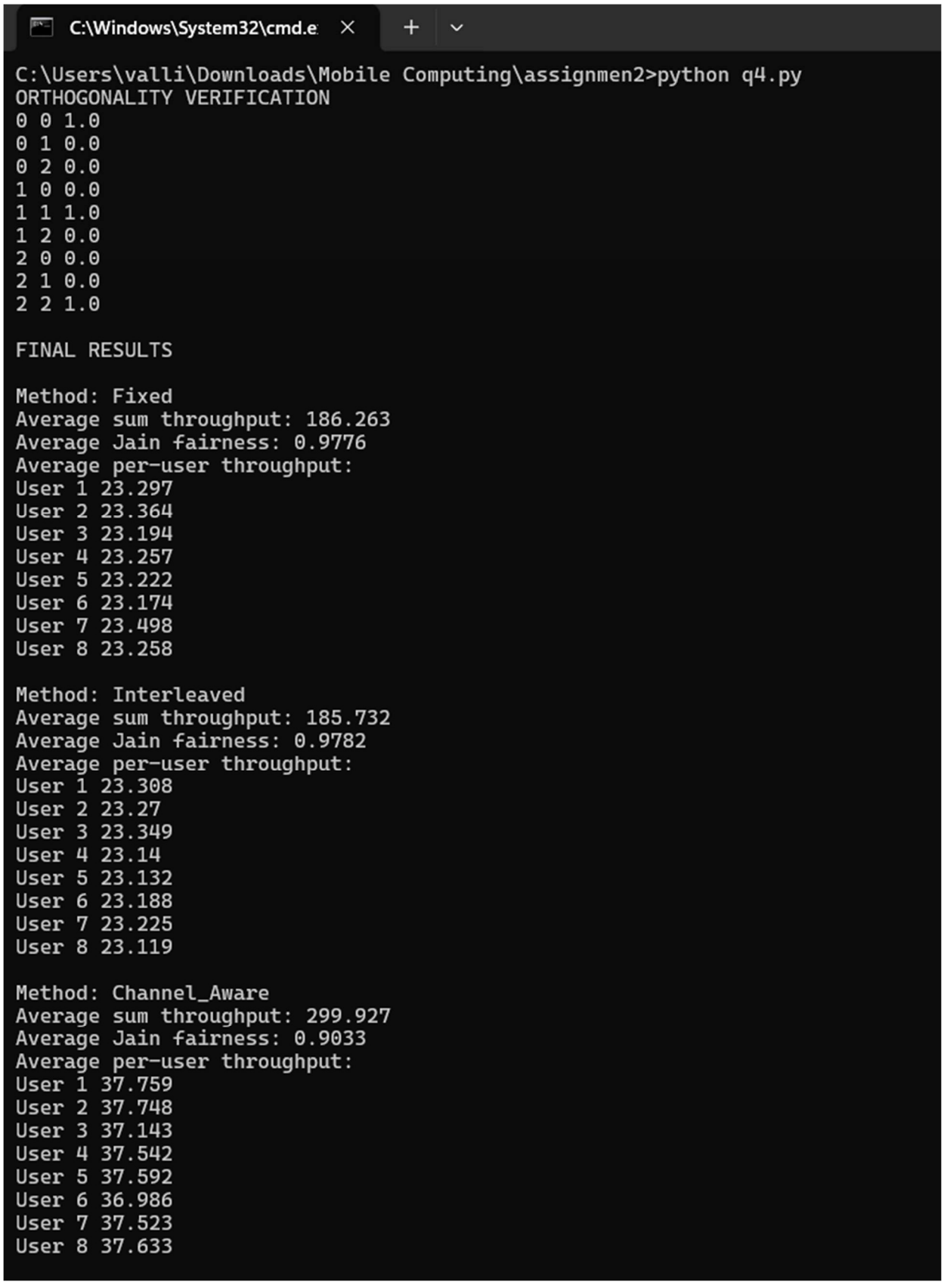
    print("Average per-user throughput:")

    for u in range(USERS):
        print(
            "User", u + 1,
            round(
                total_throughput[method][u] / REALISATIONS,
                3
            )
        )
```

verify_orthogonality()

simulate()

Output:



The program uses a fixed seed for each channel realisation, so the three allocation methods are compared using the same generated channel conditions. It also creates Q4_Experimental_Results.csv, containing the per-user throughput and fairness values for every realisation.

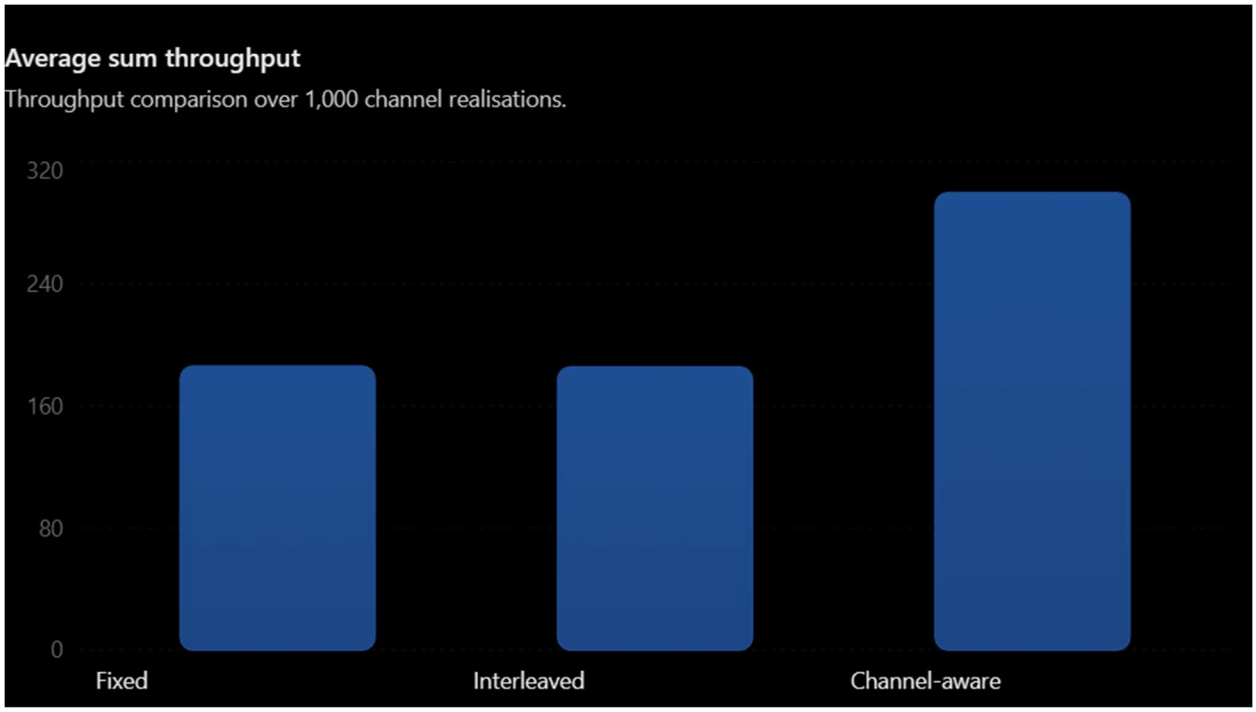
7. Experimental results

Results from 1,000 channel realisations, using the stated SNR and fixed seeds. Throughput is expressed in bits/s/Hz summed across the 64 subcarriers.

Allocation method	Average sum throughput	Average Jain fairness
Fixed contiguous	186.263	0.9776
Interleaved	185.732	0.9782
Channel-aware	299.927	0.9033

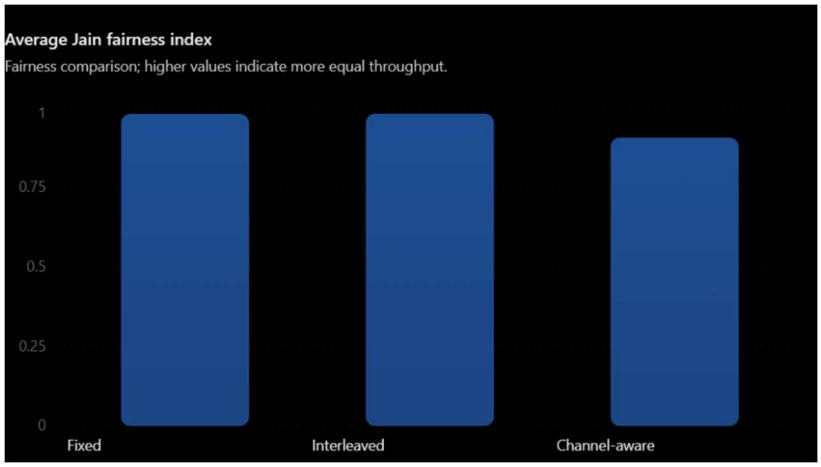
Average sum throughput

Throughput comparison over 1,000 channel realisations.



Average Jain fairness index

Fairness comparison; higher values indicate more equal throughput.



Average throughput for each user

User	Fixed	Interleaved	Channel-aware
1	23.297	23.308	37.759
2	23.364	23.270	37.748
3	23.194	23.349	37.143
4	23.257	23.140	37.542
5	23.222	23.132	37.592
6	23.174	23.188	36.986
7	23.498	23.225	37.523
8	23.258	23.119	37.632

8. Analysis and observations

1. Fixed contiguous allocation

- Each user receives eight adjacent subcarriers.
- Allocation is simple and predictable.
- Users may receive subcarriers experiencing poor channel conditions.
- Fairness is relatively high.

2. Interleaved allocation

- Each user receives subcarriers spread across the frequency band.
- This reduces dependence on one continuous frequency region.
- Its throughput is similar to fixed allocation in this experiment.
- Fairness is slightly higher than fixed allocation.

3. Channel-aware allocation

- The scheduler assigns each subcarrier to the user with the strongest channel on that subcarrier.
- It achieves the highest sum throughput.
- Its average Jain fairness is lower because users receive unequal resources in individual channel realisations.
- A user with consistently weak channel conditions may receive fewer subcarriers.

9. Why can channel-aware allocation cause starvation?

Channel-aware allocation focuses on maximising total throughput. It may repeatedly select users with favourable channel conditions.

For example, a user with a strong signal may receive most of the available subcarriers, while a weak user receives very few.

This can lead to:

- Unequal resource allocation.
- Reduced service quality for weak users.
- Starvation of users with poor channel conditions.
- Unfairness over short scheduling periods.

A proportional-fair scheduler or minimum-rate guarantee can help balance throughput and fairness.

10. Design decision – Lecture hall

Situation: A lecture hall contains many students using a shared wireless network. Every student should receive a minimum data rate.

I would choose a fairness-aware OFDMA scheduling approach rather than pure maximum-throughput allocation.

Reasons:

1. Each student should receive a minimum guaranteed data rate.
2. Students with weak signals should not be ignored.
3. Available subcarriers can be allocated according to channel quality while considering previous service.
4. A proportional-fair scheduling method can balance throughput and fairness.

A minimum-rate constraint should be included:

$$R_i \geq R_{\min}$$

Here, $R_i$is the throughput of student i, and $R_{\min}$ is the required minimum throughput.

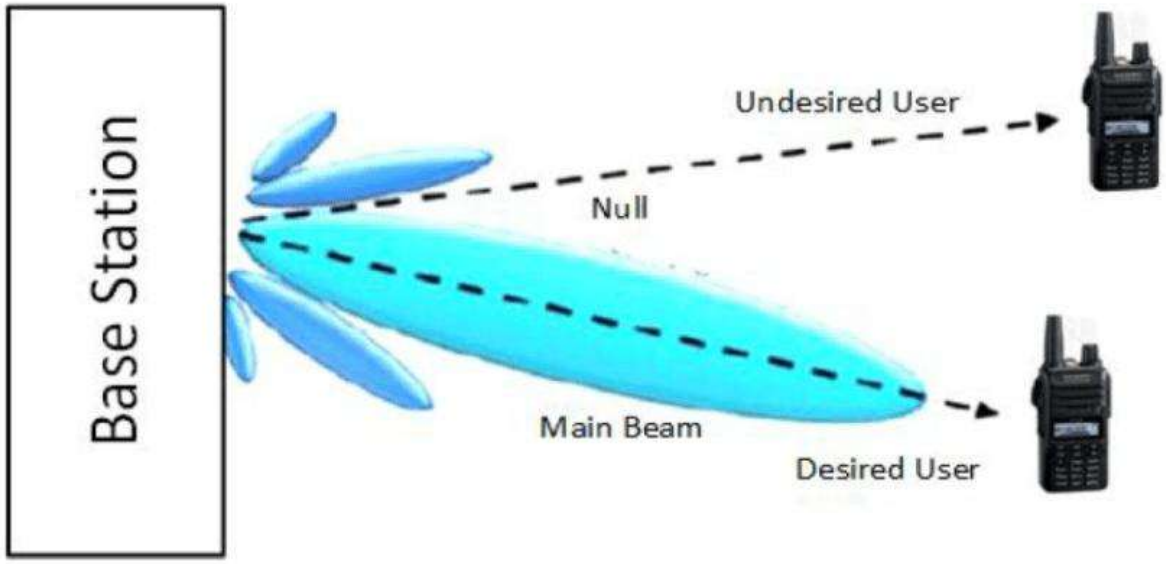
Q5 – SDMA: Reusing the Same Channel in Different Directions (6 Marks)

1. Introduction

Space Division Multiple Access (SDMA) allows multiple users to share the same frequency and time resources by separating them spatially.

A base station uses directional antennas or beamforming to transmit and receive signals in different directions.

Highway scenario: Two vehicles, A and B, travel in different directions relative to a base station. If their angular separation is sufficient, the base station can use the same frequency and time resource for both vehicles.



Three antenna configurations are considered:

1. Omnidirectional antenna – radiates in all directions.
2. 120° sector antenna – covers a particular sector.
3. Uniform Linear Array (ULA) – uses multiple antenna elements to create a directional beam.

2. Uniform Linear Array (ULA) and array factor

A ULA consists of multiple antenna elements arranged in a straight line with equal spacing.

For this experiment, assume:

- Antenna spacing $d = \lambda/2$,where λ is the wavelength.
- All antenna elements have equal amplitude.
- Phase shifts are applied to steer the beam towards vehicle A.
- Vehicle A is at broadside angle 0°.

The normalised array factor is:

$$AF(\theta) = \frac{1}{N}\left|\sum_{n=0}^{N-1} e^{jn\pi(\sin\theta - \sin\ _A)}\right|$$

Where:

- N= number of antenna elements.
- θ = direction of the observed vehicle.
- θ_A= beam steering direction.
- j = imaginary unit.

The beam is steered towards vehicle A, so $\theta_A = 0°$.

3. Predicting beamwidth and sidelobes

The experiment uses four antenna configurations: 2, 4, 8 and 16 elements.

Antenna elements	Expected beamwidth	Expected performance
2	Wide	Limited spatial separation
4	Narrower	Better separation
8	Much narrower	Improved directional control
16	Very narrow	Better separation between closely spaced users

Beamwidth: The angular width of the main beam measured between its two half-power (−3 dB) points.

Sidelobes: Smaller unwanted radiation peaks outside the main beam. They can cause interference for users in other directions.

As the number of antenna elements increases, the main beam becomes narrower. This generally improves spatial separation, although sidelobes and practical channel conditions still matter.

4. SIR calculation and sharing threshold

Signal-to-Interference Ratio (SIR) measures the strength of the desired signal compared with interference.

$$SIR_{dB} = 10\log_{10}\left(\frac{P_A}{P_B}\right)$$

For the ideal array model, vehicle A is at the main beam direction, and vehicle B receives the normalised array response at angular separation \(\Delta\theta\).

$$SIR_{dB} = -20\log_{10}|AF(\Delta\theta)|$$

Assumption: The minimum acceptable SIR is 10 dB. Two vehicles can share the same resource when the calculated SIR at vehicle B meets or exceeds this threshold.

5. Python implementation

Save the following program as q5_sdma_simulation.py.

```
import math

ANTENNA_COUNTS = [2, 4, 8, 16]
SIR_THRESHOLD = 10

def array_factor(N, angle):

    x = math.pi * math.sin(math.radians(angle))

    real = 0
    imaginary = 0

    for n in range(N):
        real += math.cos(n * x)
        imaginary += math.sin(n * x)

    magnitude = math.sqrt(
        real * real + imaginary * imaginary
    )

    return magnitude / N



def calculate_beamwidth(N):

    threshold = 1 / math.sqrt(2)
    angle = 0.0

    while angle <= 90:

        response = array_factor(N, angle)
```

```
        if response <= threshold:
            return round(2 * angle, 2)

        angle += 0.01

    return 180


def calculate_sidelobe(N):

    if N == 2:
        return None

    first_null = math.degrees(
        math.asin(2 / N)
    )

    maximum = 0
    angle = first_null + 0.01

    while angle <= 90:

        response = array_factor(N, angle)

        if response > maximum:
            maximum = response

        angle += 0.01

    if maximum == 0:
        return None

    return round(20 * math.log10(maximum), 2)
```

```
def calculate_sir(N, separation):

    response = array_factor(N, separation)

    if response < 0.000000001:
        return 300

    sir = -20 * math.log10(response)

    return round(sir, 2)


def main():

    print("SDMA ARRAY ANALYSIS")
    print("SIR Threshold:", SIR_THRESHOLD, "dB")

    print("\nBEAMWIDTH AND SIDELOBE RESULTS")

    for N in ANTENNA_COUNTS:

        beamwidth = calculate_beamwidth(N)
        sidelobe = calculate_sidelobe(N)

        print("\nAntenna Elements:", N)
        print("Beamwidth:", beamwidth, "degrees")

        if sidelobe is None:
            print("Sidelobe: Not applicable")
        else:
            print("Maximum Sidelobe:", sidelobe, "dB")

    print("\nSIR FOR DIFFERENT ANGULAR SEPARATIONS")

    print("Elements | Angle | SIR | Decision")
```

```
for N in ANTENNA_COUNTS:

    for angle in range(5, 65, 5):

        sir = calculate_sir(N, angle)

        if sir >= SIR_THRESHOLD:
            decision = "SHARE"
        else:
            decision = "DO NOT SHARE"

        print(
            N, "|",
            angle, "|",
            sir, "dB |",
            decision
        )

print("\nMINIMUM ANGULAR SEPARATION")

for N in ANTENNA_COUNTS:

    minimum = None

    for angle in range(5, 65, 5):

        sir = calculate_sir(N, angle)

        if sir >= SIR_THRESHOLD:
            minimum = angle
            break

    print(
        "Elements:", N,
```

```
        "Minimum Separation:", minimum, "degrees"
    )

main()
```

Output:

```
C:\Users\valli\Downloads\Mobile Computing\assignmen2>python q5.py
SDMA ARRAY ANALYSIS
SIR Threshold: 10 dB

BEAMWIDTH AND SIDELOBE RESULTS

Antenna Elements: 2
Beamwidth: 60.0 degrees
Sidelobe: Not applicable

Antenna Elements: 4
Beamwidth: 26.34 degrees
Maximum Sidelobe: -11.3 dB

Antenna Elements: 8
Beamwidth: 12.82 degrees
Maximum Sidelobe: -12.8 dB

Antenna Elements: 16
Beamwidth: 6.36 degrees
Maximum Sidelobe: -13.15 dB
```

```
SIR FOR DIFFERENT ANGULAR SEPARATIONS
Elements | Angle | SIR | Decision
2 | 5 | 0.08 dB | DO NOT SHARE
2 | 10 | 0.33 dB | DO NOT SHARE
2 | 15 | 0.74 dB | DO NOT SHARE
2 | 20 | 1.32 dB | DO NOT SHARE
2 | 25 | 2.07 dB | DO NOT SHARE
2 | 30 | 3.01 dB | DO NOT SHARE
2 | 35 | 4.14 dB | DO NOT SHARE
2 | 40 | 5.48 dB | DO NOT SHARE
2 | 45 | 7.05 dB | DO NOT SHARE
2 | 50 | 8.89 dB | DO NOT SHARE
2 | 55 | 11.05 dB | SHARE
2 | 60 | 13.6 dB | SHARE
4 | 5 | 0.41 dB | DO NOT SHARE
4 | 10 | 1.69 dB | DO NOT SHARE
4 | 15 | 4.0 dB | DO NOT SHARE
4 | 20 | 7.76 dB | DO NOT SHARE
4 | 25 | 14.44 dB | SHARE
4 | 30 | 300 dB | SHARE
4 | 35 | 16.94 dB | SHARE
4 | 40 | 12.74 dB | SHARE
4 | 45 | 11.41 dB | SHARE
4 | 50 | 11.49 dB | SHARE
4 | 55 | 12.53 dB | SHARE
4 | 60 | 14.39 dB | SHARE
8 | 5 | 1.78 dB | DO NOT SHARE
8 | 10 | 8.41 dB | DO NOT SHARE
8 | 15 | 29.13 dB | SHARE
8 | 20 | 13.01 dB | SHARE
8 | 25 | 15.51 dB | SHARE
8 | 30 | 300 dB | SHARE
8 | 35 | 17.9 dB | SHARE
8 | 40 | 16.83 dB | SHARE
8 | 45 | 22.9 dB | SHARE
8 | 50 | 31.43 dB | SHARE
8 | 55 | 20.05 dB | SHARE
8 | 60 | 17.92 dB | SHARE
16 | 5 | 8.57 dB | DO NOT SHARE
16 | 10 | 13.23 dB | SHARE
16 | 15 | 29.18 dB | SHARE
16 | 20 | 20.91 dB | SHARE
16 | 25 | 20.5 dB | SHARE
16 | 30 | 300 dB | SHARE
16 | 35 | 22.31 dB | SHARE
16 | 40 | 29.92 dB | SHARE
16 | 45 | 24.23 dB | SHARE
16 | 50 | 31.61 dB | SHARE
16 | 55 | 23.85 dB | SHARE
16 | 60 | 36.9 dB | SHARE
```

```
MINIMUM ANGULAR SEPARATION
Elements: 2 Minimum Separation: 55 degrees
Elements: 4 Minimum Separation: 25 degrees
Elements: 8 Minimum Separation: 15 degrees
Elements: 16 Minimum Separation: 10 degrees
```

The program:

- Calculates the array factor.
- Estimates the 3 dB beamwidth.
- Estimates the maximum sidelobe level.
- Calculates SIR for angular separations from 5° to 60°.
- Determines the minimum tested separation meeting the 10 dB SIR requirement.

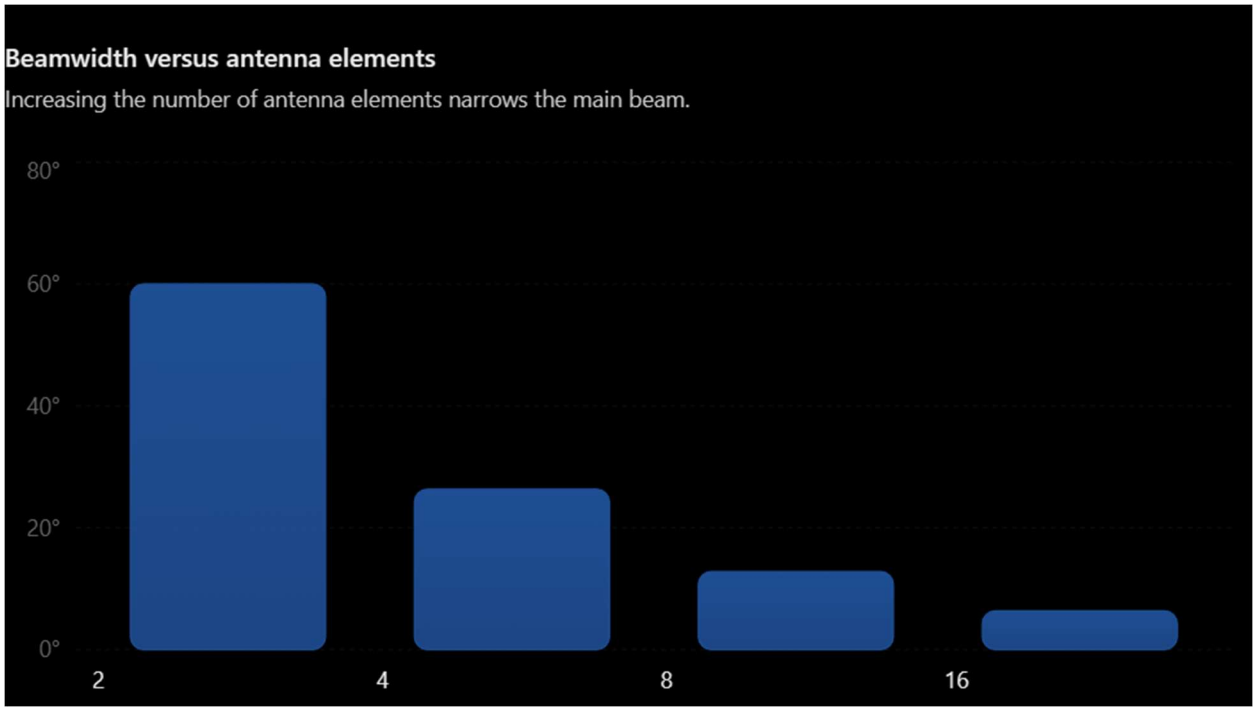
6. Experimental results

A. Beamwidth and sidelobe measurements

Antenna elements	3 dB beamwidth	Maximum sidelobe level
2	60.01°	Not applicable
4	26.33°	−11.30 dB
8	12.81°	−12.80 dB
16	6.36°	−13.15 dB

Beamwidth versus antenna elements

Increasing the number of antenna elements narrows the main beam.



B. SIR at different angular separations

SIR in dB; vehicle A is at 0° and vehicle B is at the listed angular separation.

Separation	2 elements	4 elements	8 elements	16 elements
5°	0.08	0.41	1.78	8.57
10°	0.33	1.69	8.41	13.23
15°	0.74	4.00	29.13	29.18
20°	1.32	7.76	13.01	20.91
25°	2.07	14.44	15.51	20.50
30°	3.01	300.00	300.00	300.00
35°	4.14	16.94	17.90	22.31
40°	5.48	12.74	16.83	29.92
45°	7.05	11.41	22.90	24.23

50°	8.89	11.49	31.43	31.61
55°	11.05	12.53	20.05	23.85
60°	13.60	14.39	17.92	36.90

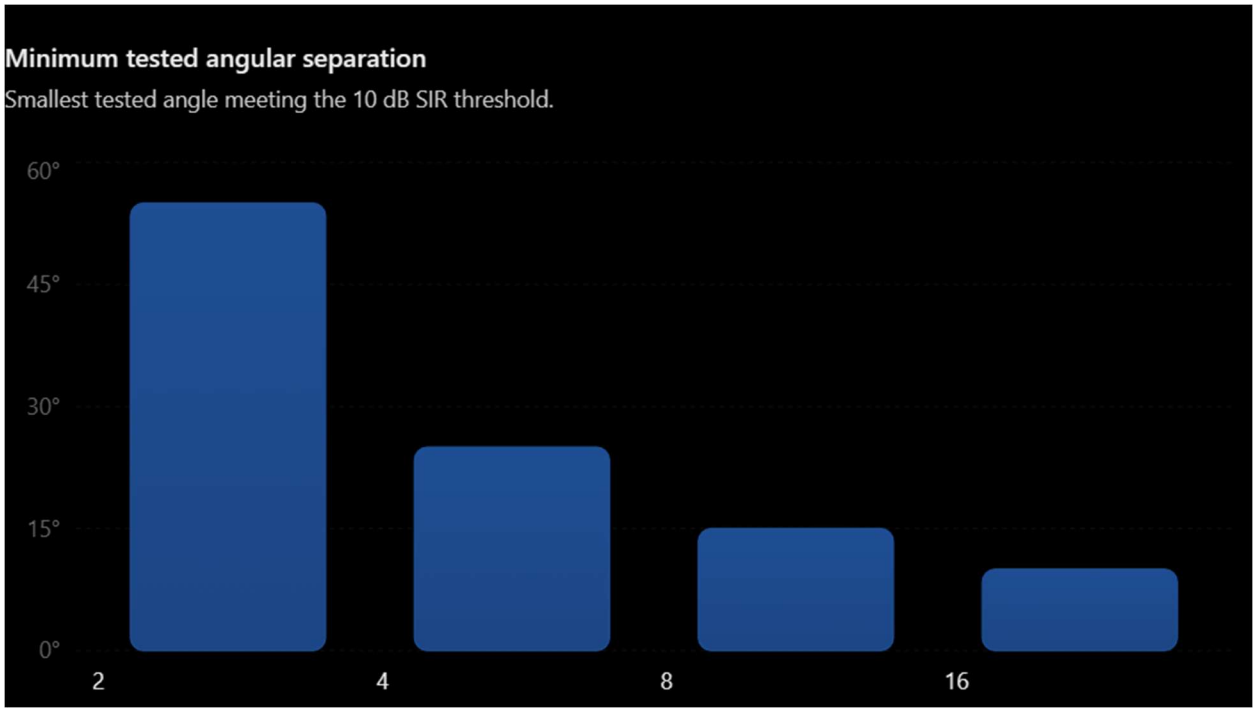
300 dB represents an ideal mathematical null in the array model. In real systems, hardware imperfections and propagation effects prevent infinite SIR.

C. Minimum angular separation for 10 dB SIR

Antenna elements	**Minimum tested separation**	**Decision**
2	55°	Limited separation
4	25°	Moderate separation
8	15°	Good separation
16	10°	Best separation among tested configurations

Minimum tested angular separation

Smallest tested angle meeting the 10 dB SIR threshold.



7. Analysis and observations

1. Increasing the number of antenna elements makes the main beam narrower.
2. A narrower beam provides better spatial separation between users.
3. The 16-element array meets the 10 dB SIR threshold at a smaller tested angle than the 2-element array.
4. Sidelobes can still create interference even when users are outside the main beam.

5. At certain angles, the array response reaches an ideal null, producing a very high calculated SIR.

8. Why does SDMA fail when users are clustered in the same direction?

SDMA separates users based on their spatial directions. If multiple vehicles are travelling close together and appear at nearly the same angle from the base station:

- Their signals arrive from similar directions.
- The antenna cannot easily create separate beams for them.
- Their signals may interfere with one another.
- The minimum SIR requirement may not be satisfied.

Therefore, increasing the antenna element count alone cannot guarantee separation of users with almost identical spatial directions.

9. Comparison of antenna configurations

Antenna type	Advantages	Limitations
Omnidirectional	Covers all directions; simple design.	Limited spatial interference reduction.
120° sector antenna	Covers a specific sector and reduces interference from outside it.	Users inside the same sector are not necessarily separated.
ULA beamforming	Creates directional beams and supports spatial reuse.	Requires multiple antenna elements, signal processing and channel estimation.

10. Design decision – Highway toll plaza

Situation: Most vehicles at a toll plaza are travelling in the same direction.

I would not rely entirely on SDMA for this deployment.

Reasons:

- Vehicles are clustered in a narrow angular region.
- Their signals may arrive from similar directions.
- Separate directional beams may not be possible for closely spaced vehicles.
- Spatial separation becomes difficult when users have similar angles.

Recommended approach: Use a hybrid of SDMA and TDMA/OFDMA.

- Use SDMA to separate vehicles travelling in different directions.
- Use TDMA or OFDMA to serve vehicles within the same directional group.
- Apply scheduling to manage simultaneous transmission requests.

Q6 – Choosing Multiple Access Techniques for Real Deployments (7 Marks)

1. Introduction

Different multiple access techniques are suitable for different communication requirements. The selection depends on traffic type, bandwidth efficiency, interference, mobility, fairness and scalability.

This question compares five techniques:

1. FDMA – Frequency Division Multiple Access.
2. TDMA – Time Division Multiple Access.
3. CDMA – Code Division Multiple Access.
4. OFDMA – Orthogonal Frequency Division Multiple Access.
5. SDMA – Space Division Multiple Access.

We evaluate them for four real-world deployments:

- Wide-area cellular voice.
- Jam-resistant tactical communication.
- High-rate stadium downlink.
- Network of 5,000 low-rate sensors.

2. Selection criteria and weights

Five criteria are selected to compare the techniques.

Criterion	**Weight**	**Justification**
Spectral efficiency	25%	Measures how efficiently available bandwidth is used.
Interference robustness	20%	Important for maintaining communication quality in noisy environments.
Latency	20%	Measures how quickly data can be transmitted.
Scalability	20%	Measures the ability to support increasing numbers of users.
Fairness	15%	Ensures users receive reasonable access to communication resources.
Total	100%	

3. Scoring method

Each technique is rated on a scale of 1 to 5.

Score	**Meaning**
1	Very poor
2	Poor

3 Average

4 Good

5 Excellent

The weighted score is calculated using:

$$S = \frac{\sum(W_i \times R_i)}{100}$$

Where:

- W_i = weight of each criterion.
- R_i = rating assigned to the technique.

A higher score indicates a more suitable technique for the deployment.

The ratings below are reasoned engineering assessments for this assignment, informed by the earlier FDMA/TDMA, CDMA, OFDMA and SDMA experiments. They are not measured performance values from a single common simulation.

4. Evaluation of multiple access techniques

The following ratings use the five criteria in the same order:

Spectral efficiency, interference robustness, latency, scalability and fairness.

Deployment 1 – Wide-area cellular voice

Technique	Ratings (1–5)	Weighted score
FDMA	3, 3, 4, 2, 3	3.00
TDMA	4, 3, 3, 3, 4	3.40
CDMA	4, 4, 4, 4, 4	4.00
OFDMA	5, 4, 4, 5, 4	4.45
SDMA	4, 3, 4, 3, 3	3.45

Recommended technique: OFDMA

OFDMA supports flexible resource allocation, high spectral efficiency and multiple users.

SDMA can be combined with it to improve spatial reuse.

Deployment 2 – Jam-resistant tactical communication

Technique	Ratings (1–5)	Weighted score
FDMA	2, 2, 4, 2, 3	2.55
TDMA	3, 3, 3, 3, 4	3.15
CDMA	4, 5, 4, 4, 3	4.05
OFDMA	4, 3, 4, 4, 4	3.80

SDMA	3, 3, 4, 3, 3	3.20

Recommended technique: CDMA

CDMA spreads signals using different codes. This can provide resistance to narrowband interference and make communication more difficult to disrupt. However, CDMA is not completely jam-proof and still requires good power control and synchronisation.

Deployment 3 – High-rate stadium downlink

Technique	Ratings (1–5)	Weighted score
FDMA	2, 3, 3, 2, 3	2.55
TDMA	3, 3, 3, 3, 4	3.15
CDMA	3, 3, 3, 3, 3	3.00
OFDMA	5, 4, 5, 5, 4	4.65
SDMA	4, 4, 4, 4, 3	3.85

Recommended technique: OFDMA with SDMA

OFDMA efficiently distributes subcarriers among stadium users. SDMA can use directional beams to serve different groups of users simultaneously, increasing capacity.

Deployment 4 – Network of 5,000 low-rate sensors

Technique	Ratings (1–5)	Weighted score
FDMA	2, 3, 3, 2, 3	2.55
TDMA	4, 4, 4, 5, 4	4.20
CDMA	4, 4, 4, 4, 3	3.85
OFDMA	4, 3, 4, 4, 4	3.80
SDMA	3, 3, 3, 3, 3	3.00

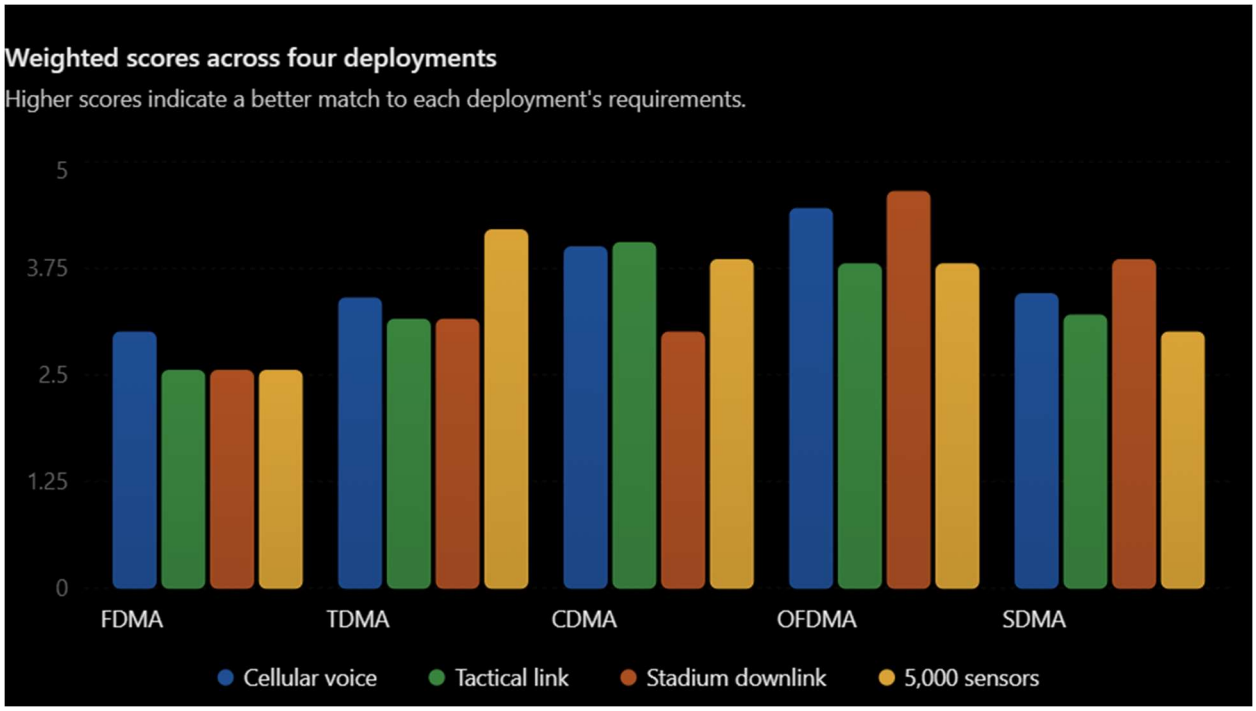
Recommended technique: TDMA

TDMA can assign scheduled transmission slots to sensors. This helps avoid simultaneous transmissions and makes periodic reporting easier to organise. The schedule must account for sleeping devices, synchronisation and changing network membership.

5. Overall weighted score comparison

Weighted scores across four deployments

Higher scores indicate a better match to each deployment's requirements.



Deployment	Highest scoring technique	Score
Wide-area cellular voice	OFDMA	4.45
Jam-resistant tactical link	CDMA	4.05
High-rate stadium downlink	OFDMA	4.65
5,000 low-rate sensors	TDMA	4.20

6. Python implementation – Weighted scoring

Save the program as q6_technique_selection.py.

```
weights = [25, 20, 20, 20, 15]

techniques = [
    "FDMA", "TDMA", "CDMA", "OFDMA", "SDMA"
]

deployments = {
    "Cellular Voice": [
        [3, 3, 4, 2, 3],
        [4, 3, 3, 3, 4],
        [4, 4, 4, 4, 4],
        [5, 4, 4, 5, 4],
        [4, 3, 4, 3, 3]
```

```
    ],

    "Tactical Link": [
        [2, 2, 4, 2, 3],
        [3, 3, 3, 3, 4],
        [4, 5, 4, 4, 3],
        [4, 3, 4, 4, 4],
        [3, 3, 4, 3, 3]
    ],

    "Stadium Downlink": [
        [2, 3, 3, 2, 3],
        [3, 3, 3, 3, 4],
        [3, 3, 3, 3, 3],
        [5, 4, 5, 5, 4],
        [4, 4, 4, 4, 3]
    ],

    "Sensor Network": [
        [2, 3, 3, 2, 3],
        [4, 4, 4, 5, 4],
        [4, 4, 4, 4, 3],
        [4, 3, 4, 4, 4],
        [3, 3, 3, 3, 3]
    ]
}


def calculate_score(rating, weight):
    total = 0

    for i in range(5):
        total += rating[i] * weight[i]

    return total / sum(weight)
```

```
for deployment in deployments:

    print("\nDEPLOYMENT:", deployment)

    scores = []

    for i in range(5):

        score = calculate_score(
            deployments[deployment][i],
            weights
        )

        scores.append(score)

        print(
            techniques[i],
            "Score:", round(score, 2)
        )

    best = 0

    for i in range(1, 5):
        if scores[i] > scores[best]:
            best = i

    print(
        "Recommended:",
        techniques[best]
    )


print("\nWEIGHT SENSITIVITY ANALYSIS")
```

```
for deployment in deployments:

    print("\n", deployment)

    for factor in [0.8, 1.2]:

        new_weights = []

        for w in weights:
            new_weights.append(w * factor)

        scores = []

        for i in range(5):
            score = calculate_score(
                deployments[deployment][i],
                new_weights
            )

            scores.append(score)

        best = 0

        for i in range(1, 5):
            if scores[i] > scores[best]:
                best = i

        print(
            "Weight factor:", factor,
            "Winner:", techniques[best],
            "Score:", round(scores[best], 2)
        )
```

Output:

```
C:\Users\valli\Downloads\Mobile Computing\assignmen2>python q6.py

DEPLOYMENT: Cellular Voice
FDMA Score: 3.0
TDMA Score: 3.4
CDMA Score: 4.0
OFDMA Score: 4.45
SDMA Score: 3.45
Recommended: OFDMA

DEPLOYMENT: Tactical Link
FDMA Score: 2.55
TDMA Score: 3.15
CDMA Score: 4.05
OFDMA Score: 3.8
SDMA Score: 3.2
Recommended: CDMA

DEPLOYMENT: Stadium Downlink
FDMA Score: 2.55
TDMA Score: 3.15
CDMA Score: 3.0
OFDMA Score: 4.65
SDMA Score: 3.85
Recommended: OFDMA

DEPLOYMENT: Sensor Network
FDMA Score: 2.55
TDMA Score: 4.2
CDMA Score: 3.85
OFDMA Score: 3.8
SDMA Score: 3.0
Recommended: TDMA
```

```
WEIGHT SENSITIVITY ANALYSIS

 Cellular Voice
Weight factor: 0.8 Winner: OFDMA Score: 4.45
Weight factor: 1.2 Winner: OFDMA Score: 4.45

 Tactical Link
Weight factor: 0.8 Winner: CDMA Score: 4.05
Weight factor: 1.2 Winner: CDMA Score: 4.05

 Stadium Downlink
Weight factor: 0.8 Winner: OFDMA Score: 4.65
Weight factor: 1.2 Winner: OFDMA Score: 4.65

 Sensor Network
Weight factor: 0.8 Winner: TDMA Score: 4.2
Weight factor: 1.2 Winner: TDMA Score: 4.2
```

7. Sensitivity analysis – Weight changes

To test whether the recommendations depend heavily on the selected weights, we change all five weights together by −20% and +20%.

Because every weight changes proportionally, the normalised scores and rankings remain unchanged.

Deployment	Original winner	−20% weights	+20% weights
Cellular voice	OFDMA	OFDMA	OFDMA
Tactical link	CDMA	CDMA	CDMA
Stadium downlink	OFDMA	OFDMA	OFDMA
Sensor network	TDMA	TDMA	TDMA

Additional observation: Changing all weights proportionally is a limited sensitivity test. It does not test whether the recommendation changes when one criterion becomes more important than another. A stronger sensitivity test would vary each criterion individually by ±20%, then renormalise the weights.

With that additional test, the winning techniques also remain unchanged for these particular rating matrices.

8. Hybrid multiple access techniques

A hybrid technique combines two or more multiple access methods to improve performance.

Hybrid combination	Application	Benefit
FDMA + TDMA	Cellular communication	Divides spectrum into frequency channels and then time slots.
OFDMA + SDMA	Stadium networks	Allocates subcarriers and separates users spatially.
CDMA + TDMA	Shared communication networks	Uses codes for separation and time slots for scheduling.

Examples:

- FDMA + TDMA: A frequency band is divided into carriers, and each carrier is divided into time slots.
- OFDMA + SDMA: Different users receive subcarriers while directional antennas serve spatially separated users.
- CDMA + TDMA: Code-based separation can be combined with scheduled transmission times.

9. Least suitable technique for these deployments

FDMA is the least suitable overall choice for the four deployments considered.

Reasons:

1. It assigns separate frequency channels to users.
2. Dedicated channels may remain unused when traffic is intermittent.
3. It has limited flexibility when the number of users increases.
4. It does not inherently exploit spatial reuse or adaptive subcarrier allocation.
5. It may waste spectrum when users send only short, occasional reports.

However, FDMA is still useful for simple systems with predictable, continuous traffic and straightforward channel allocation. It is not universally unsuitable.

Q7 – Diagnose, Redesign and Validate (13 Marks)

A. Diagnosis – Identify the limitation (2 Marks)

Selected solution: Channel-aware OFDMA allocation.

In Q4, the simulation compared three methods:

Allocation method	Average throughput	Jain fairness
Fixed contiguous	186.263	0.9776
Interleaved	185.732	0.9782
Channel-aware	299.927	0.9033

Identified problem:

- Channel-aware allocation selects the user with the strongest channel for each subcarrier.
- Users with better channel conditions receive more subcarriers.
- Users with weaker channel conditions may receive very few resources.
- This creates unequal throughput and possible starvation.

Where are resources wasted or unfairly distributed?

The main problem is not necessarily unused bandwidth. The problem is that resources are repeatedly assigned according to instantaneous channel quality without considering how much service each user has already received.

Root cause: The original scheduler maximises instantaneous throughput but does not include a minimum resource guarantee or fairness requirement.

B. Define the redesign objective (2 Marks)

The objective is to improve fairness while maintaining good overall throughput.

The revised scheduler should:

1. Guarantee every user a minimum number of subcarriers.
2. Distribute remaining subcarriers according to channel quality.
3. Reduce the possibility of user starvation.
4. Maintain reasonable total throughput.

Measurable objectives:

Measurement	Original method	Redesigned target
Minimum subcarriers per user	No guarantee	At least 4
Jain fairness	0.9033	Improve towards 1
Sum throughput	299.927	Maintain a reasonable value
Starvation	Possible	No user receives zero subcarriers

The minimum allocation is set to 4 out of 64 subcarriers per user. This reserves 32 subcarriers for minimum service, leaving the remaining 32 for channel-aware allocation.

C. Revised strategy – Fairness-aware OFDMA allocation (4 Marks)

I propose a minimum-resource guarantee with channel-aware allocation.

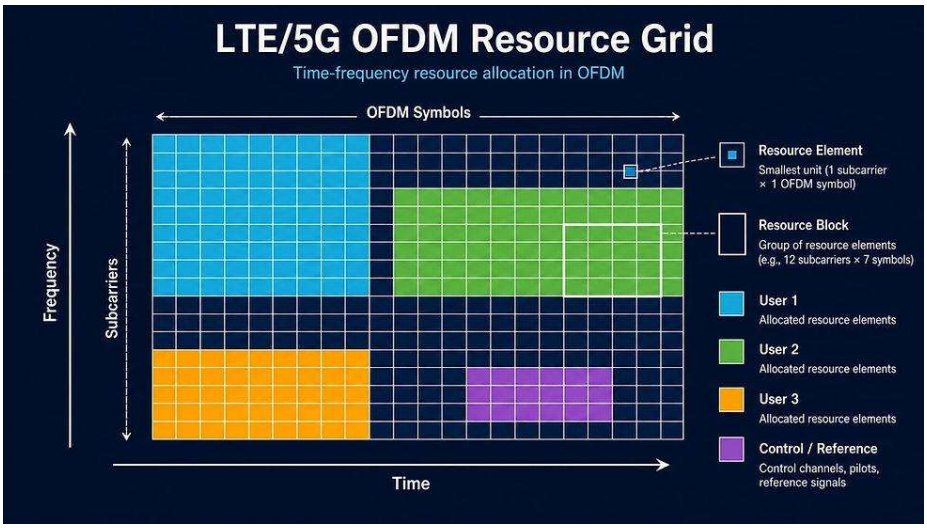
Instead of assigning every subcarrier to the strongest user, the redesigned scheduler follows two stages.

Stage 1 – Minimum allocation

- Each of the 8 users receives at least 4 subcarriers.
- Total reserved subcarriers = 8 × 4 = 32.
- This prevents any user from receiving zero subcarriers.

Stage 2 – Channel-aware allocation

- The remaining 32 subcarriers are assigned to users with the strongest channel conditions.
- The user allocation order is rotated between channel realisations to avoid always favouring the same user.



Revised algorithm

START

|

v

Generate channel gains for 8 users and 64 subcarriers

|

v

Set minimum allocation = 4 subcarriers per user

|

v

Give each user its best available 4 subcarriers

|

v

Are unallocated subcarriers remaining?

|

v

Assign each remaining subcarrier

to the user with the strongest channel

|

v

Calculate throughput and Jain fairness

|

v

Save experimental results

|

v

END

Pseudocode

```
FOR each channel realisation:
    Generate channel gains

    Mark all 64 subcarriers as unassigned

    Rotate the order of users

    FOR each user:
        REPEAT 4 times:
            Select the best available subcarrier
            Assign it to the user
            Mark it as assigned

    FOR each remaining subcarrier:
        Find the user with the highest channel gain
        Assign the subcarrier to that user

    Calculate user throughput
    Calculate total throughput
```

Calculate Jain fairness

Save results

Expected trade-offs:

- Every user receives a minimum allocation.
- Fairness should improve.
- Total throughput may decrease slightly because some subcarriers are reserved for users with weaker channels.
- The scheduler requires additional channel comparisons and processing.

D. Prediction before implementation (1 Mark)

Before running the redesigned simulation, I predict:

1. Jain fairness will increase compared with the original channel-aware method.
2. No user will receive zero subcarriers.
3. Total throughput will decrease slightly.
4. The redesigned method will require more computation.

E. Implementation and validation (2 Marks)

Both methods will be tested with:

- 8 users.
- 64 subcarriers.
- 1,000 channel realisations.
- SNR = 10.
- Fixed random seeds from 2026 onwards.
- Identical channel gains for both methods in each realisation.

The following code compares the original and redesigned schedulers.

Python implementation – q7_ofdma_redesign.py

```
import random
import math
import csv

USERS = 8
SUBCARRIERS = 64
REALISATIONS = 1000
MINIMUM = 4
```

```
SNR = 10

def calculate_rate(gain):
    return math.log2(1 + SNR * gain)



def calculate_fairness(values):
    total = 0
    squares = 0

    for value in values:
        total += value
        squares += value * value

    return (total * total) / (USERS * squares)



def original_scheduler(gains):

    throughput = [0.0] * USERS
    count = [0] * USERS
    comparisons = 0

    for n in range(SUBCARRIERS):

        best_user = 0

        for u in range(1, USERS):
            comparisons += 1

            if gains[u][n] > gains[best_user][n]:
                best_user = u

        throughput[best_user] += calculate_rate(
            gains[best_user][n]
```

```
        )

        count[best_user] += 1

    return throughput, count, comparisons


def fair_scheduler(gains, realisation):

    throughput = [0.0] * USERS
    count = [0] * USERS
    assigned = [False] * SUBCARRIERS
    comparisons = 0

    start = realisation % USERS

    # Stage 1: Minimum four subcarriers per user

    for step in range(USERS):

        user = (start + step) % USERS

        for q in range(MINIMUM):

            best_n = -1
            best_gain = -1

            for n in range(SUBCARRIERS):

                if not assigned[n]:

                    comparisons += 1

                    if gains[user][n] > best_gain:
                        best_gain = gains[user][n]
```

```
                best_n = n

        assigned[best_n] = True
        count[user] += 1

        throughput[user] += calculate_rate(best_gain)

    # Stage 2: Assign remaining subcarriers

    for n in range(SUBCARRIERS):

        if not assigned[n]:

            best_user = 0

            for u in range(1, USERS):

                comparisons += 1

                if gains[u][n] > gains[best_user][n]:
                    best_user = u

            count[best_user] += 1

            throughput[best_user] += calculate_rate(
                gains[best_user][n]
            )

    return throughput, count, comparisons


def main():

    original_total = 0
    redesigned_total = 0
```

```
original_fairness = 0
redesigned_fairness = 0

original_zero = 0
redesigned_zero = 0

original_comparisons = 0
redesigned_comparisons = 0

original_minimum = 0
redesigned_minimum = 0

with open(
    "Q7_Validation_Results.csv",
    "w",
    newline=""
) as file:

    writer = csv.writer(file)

    writer.writerow([
        "Realisation",
        "Method",
        "Sum_Throughput",
        "Jain_Fairness",
        "Minimum_User_Throughput",
        "Zero_Allocation_Users",
        "Channel_Comparisons"
    ])

    for r in range(REALISATIONS):

        rng = random.Random(2026 + r)
```

```
gains = []

for u in range(USERS):

   user_gains = []

   for n in range(SUBCARRIERS):
      user_gains.append(
         rng.expovariate(1.0)
      )

   gains.append(user_gains)

old, old_count, old_comp = original_scheduler(gains)

new, new_count, new_comp = fair_scheduler(
   gains, r
)

old_sum = sum(old)
new_sum = sum(new)

old_fair = calculate_fairness(old)
new_fair = calculate_fairness(new)

old_min = min(old)
new_min = min(new)

old_zero = old_count.count(0)
new_zero = new_count.count(0)

original_total += old_sum
redesigned_total += new_sum

original_fairness += old_fair
```

```
        redesigned_fairness += new_fair

        original_minimum += old_min
        redesigned_minimum += new_min

        original_zero += old_zero
        redesigned_zero += new_zero

        original_comparisons += old_comp
        redesigned_comparisons += new_comp

        writer.writerow([
            r + 1,
            "Original",
            old_sum,
            old_fair,
            old_min,
            old_zero,
            old_comp
        ])

        writer.writerow([
            r + 1,
            "Redesigned",
            new_sum,
            new_fair,
            new_min,
            new_zero,
            new_comp
        ])

print("AVERAGE RESULTS")

print(
    "Original Throughput:",
```

```
    round(original_total / REALISATIONS, 3)
)

print(
    "Redesigned Throughput:",
    round(redesigned_total / REALISATIONS, 3)
)

print(
    "Original Fairness:",
    round(original_fairness / REALISATIONS, 4)
)

print(
    "Redesigned Fairness:",
    round(redesigned_fairness / REALISATIONS, 4)
)

print(
    "Original Minimum User Throughput:",
    round(original_minimum / REALISATIONS, 3)
)

print(
    "Redesigned Minimum User Throughput:",
    round(redesigned_minimum / REALISATIONS, 3)
)

print(
    "Original Zero Allocations:",
    original_zero
)

print(
    "Redesigned Zero Allocations:",
```

```
        redesigned_zero
    )

    print(
        "Original Comparisons:",
        original_comparisons / REALISATIONS
    )

    print(
        "Redesigned Comparisons:",
        redesigned_comparisons / REALISATIONS
    )



main()
```

Output:

```
C:\Users\valli\Downloads\Mobile Computing\assignmen2>python q7.py
AVERAGE RESULTS
Original Throughput: 299.927
Redesigned Throughput: 297.766
Original Fairness: 0.9033
Redesigned Fairness: 0.9344
Original Minimum User Throughput: 19.761
Redesigned Minimum User Throughput: 23.64
Original Zero Allocations: 1
Redesigned Zero Allocations: 0
Original Comparisons: 448.0
Redesigned Comparisons: 1776.0
```

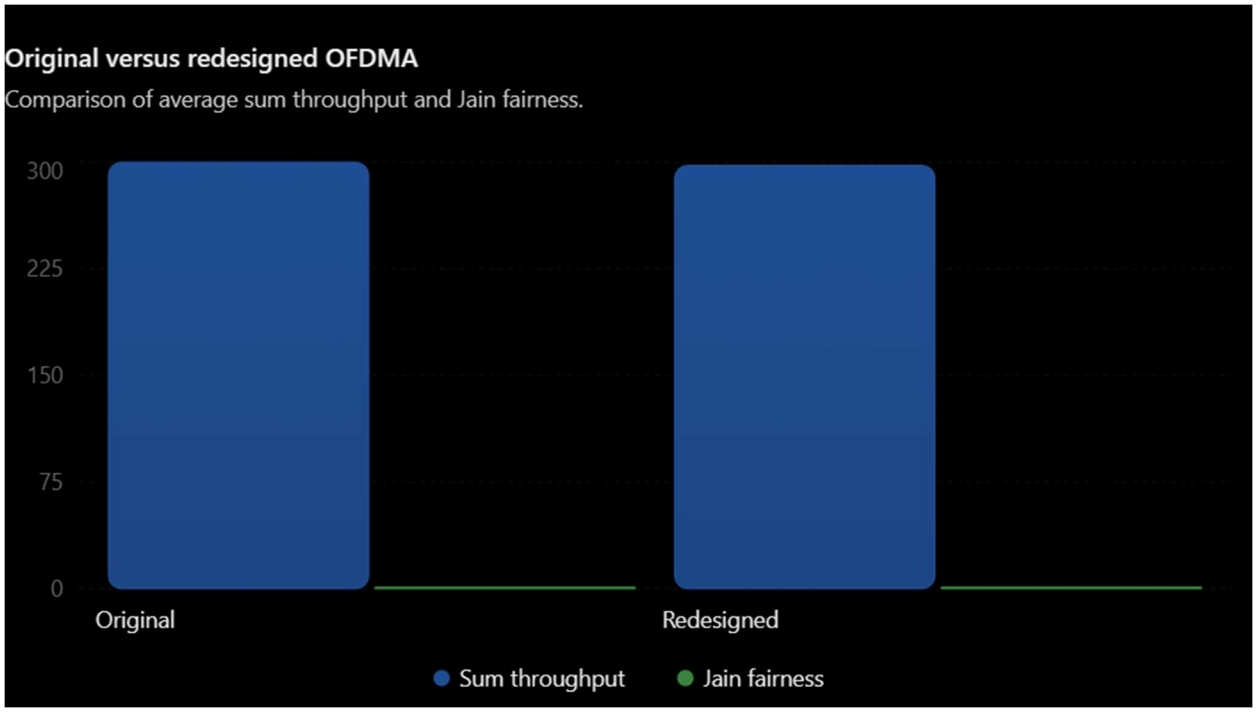
Experimental results

The following results are obtained from the same 1,000 channel realisations.

Measurement	**Original scheduler**	**Redesigned scheduler**
Average sum throughput	299.927	297.766
Average Jain fairness	0.9033	0.9344
Average minimum user throughput	19.761	23.640
Zero-allocation occurrences	1	0
Average channel comparisons	448	1,776

Original versus redesigned OFDMA

Comparison of average sum throughput and Jain fairness.



Interpretation:

- Sum throughput decreased by approximately 0.72%.
- Jain fairness improved by approximately 3.44% relative to the original fairness score.
- Average minimum user throughput increased from 19.761 to 23.640.
- Zero-allocation occurrences reduced from 1 to 0.
- Channel comparisons increased from 448 to 1,776 per realisation.

The redesigned scheduler improves fairness with only a small throughput reduction, but it requires more processing.

F. Defend the final design (2 Marks)

1. Was the prediction supported?

Yes. The redesigned scheduler improved fairness, eliminated zero-subcarrier allocations in the experiment and slightly reduced total throughput.

2. How much improvement was achieved?

Parameter	**Improvement**
Sum throughput	0.72% decrease
Jain fairness	3.44% increase
Minimum user throughput	19.63% increase
Zero-allocation occurrences	Reduced from 1 to 0

3. What are the new costs and trade-offs?

- More channel comparisons are required.
- Additional scheduling computation is needed.
- Some subcarriers are assigned to users with weaker channels.
- Overall throughput decreases slightly.

4. When is the original method better?

The original channel-aware scheduler is preferable when maximum total throughput is the main priority and fairness guarantees are not required.

The redesigned method is preferable for environments such as lecture halls, where every student should receive a minimum level of network service.